

Big Data Analytics

Stefano Lodi

Alma Mater Studiorum Università di Bologna

91262 - DATA MINING, TEXT MINING AND BIG DATA ANALYTICS (Mod. 3)

Outline

- 1 Premise
- 2 Introduction
- 3 MapReduce
- 4 Spark
- 5 Spark structured API
- 6 MLlib
- 7 Spark unstructured API

Section 1

Premise

Scope and summary of the topics

The Big Data Analytics (BDA) Module 3 is an introduction to two popular tools for BDA, **Apache Hadoop** and **Apache Spark**

Main topics

- Introduction to BDA
- Apache Hadoop, HDFS, MapReduce, virtual machine (VM) configuration
- Apache Spark, structured Application Programmer's Interface (API), unstructured API and examples of distributed programming

Instructions to configure the VM are given in lectures 1-2.

Available material and readings

The course's section on our official [course material site](#) contains a subsection dedicated to the BDA module, with slides, configuration files, examples, and few sample exam questions, downloadable by any participant. Lecture Panopto recordings will also be available on the site, 2-7 days after the lecture's date. I'll do my best to shorten the lag.

Comprehensiveness Course material and official Apache on-line documentation should suffice.

Beyond That said, there are many valuable books and resources on Hadoop and Spark. I can mention the ones I personally found useful, in case you want to read further.

- White, T. (2009). Hadoop: The definitive guide 4th Edition. Reilly Media.
- Chambers, B., & Zaharia, M. (2018). Spark: The definitive guide: Big data processing made simple. Reilly Media.

Examination

Official information on the course's two-part assessment method can be found in [the course's web page](#). Both parts are relevant to the BDA module.

Multiple choice sentence completions 3 possible completions for each sentence, only 1 correct

- Automatically graded between 0 and 30
- The fraction of BDA questions will be the same as the fraction of BDA lecture hours

Project and project discussion If you choose the BDA module for preparing your project and project discussion, please send me a detailed enough proposal, declaring the source of your data, the analysis questions you would like to answer, and the tools and algorithms you plan to use to find the answers.

Section 2

Introduction

Eras in data processing

- Until 1970: Data stored on magnetic media
 - Initially on magnetic tapes, thus allowing sequential access only
 - Later, as files on magnetic spinning disks with direct access
 - Indexing methods for accessing data by key on disks
- 1970-2000: Data managed by integrated systems
 - Exclusive support for the relational model, in most cases
 - SQL
 - Client-server communication
- Since 2000: Massive amounts of data generated by the Internet, in unpredictable forms
 - Variety of systems for data management, including relational DBMSs
 - Matching the system with the application is an additional design challenge
 - Hard to predict which systems will stand the test of time

Business, social, and the Big Data loop

- Towards the end of the relational data era, Internet emerged as the planetary wide area network
- In the Internet, hyperlinked pages written in HTML emerged as the prevalent form of data, and constituted the World Wide Web
- Such technologies gave businesses, mostly retailers, the opportunity to create E-commerce portals as a new sales channel
- When customers gained ubiquitous, anytime access to the Internet, through smartphones and faster home connections, retailers began to use both social network data, and data recorded by their own Web sites, to improve performance and extend the online market or market share, thus generating more data
- The result of such process is the Big Data loop
- The model can be similarly described for other types of business
- Ethical issues are raised, which the scientific community has **only recently started to address**

Data origins and types

The origins and types of data are diverse

- Traditional operational data
 - Company business data, such as codes of products purchased at stores and online
 - Institutional activity data
- Data automatically generated by sensors
 - Satellites
 - Environmental monitoring stations
- Scientific experiments
 - Large Hadron Collider (LHC), Geneva
- Social networking activity
 - Facebook, Instagram
- Private multimedia recordings

Data size and growth

- In all eras of data processing database size has grown exponentially
- New storage = sum of all old storage (ever)
 - cf. [Jim Gray's Turing Award talk](#) (1998)
- New York Stock Exchange about 4-5 TB/d
- Facebook 240 billion photos, 7 PB/m
- Large Hadron Collider 30 PB/y
- The 2025 projection of the total size of digital data has been estimated as exceeding 180 ZB ([source: Statista](#))

1 TB	1 terabyte	10^{12} byte
1 PB	1 petabyte	10^{15} byte
1 ZB	1 zettabyte	10^{21} byte

Complex data processing

- Moreover, we can now process the majority of, or all, such types of data in complex ways using machine learning and predictive analytics
- Complex analyses require fast access to storage, however, the progress in access speed has been slower than that in disk size
 - 1000-fold increase in size vs. 25-fold increase in access speed
- Note that, although parallelism can help, e. g., 100 drives in parallel can make up for the trailing progress of access speed, it also raises further issues of reliability with such a large number of concurrent devices, and complexity of parallel computation

- Google's activity on big data started in 1998 with the implementation of the PageRank algorithm for returning the most relevant Web pages matching a query term
- Subsequently, Google created its own hardware infrastructure, based on $\sim 10^5$ small servers, and operated using three novel software components
 - **Google File System** (GFS): A distributed file system optimised for sequential reads of, and writes that append to, large files, on a multitude of unreliable machines
 - **MapReduce**: A programming model for processing large datasets through map and reduce functions automatically in parallel on unreliable machines
 - **BigTable**: A distributed, non-relational storage system that is highly scalable in data size and number of machines
- Both MapReduce and BigTable use GFS for file storage

Hadoop

- A project which draws from Google Big Data technologies to create open-source software
- Since 2007, it has been downloadable from the [Apache website](#); it has been adopted by Yahoo! and Facebook
- Hadoop is intended to run on a large number of commodity computing nodes
 - Low cost per byte
 - Massively scalable in storage and computing power
 - Reliable: data are stored with redundancy; failure of a computing node does not crash the whole computation, the task of the failed node is reassigned to another node
 - Through MapReduce, the processing model is distributed and scalable

Hadoop ecosystem: an overview

- **HDFS** (Hadoop Distributed File System) is a distributed filesystem capable of operating reliably on a large number of servers realised with commodity, unreliable hardware
- **HBase** is a key-value (non-relational) store which uses HDFS as filesystem
- **YARN** (Yet Another Resource Negotiator) is as system for the management of resources in a cluster of computers. It allows for the execution of distributed programs in a Hadoop cluster of computers
- More recent developments include projects for stream processing, running SQL-like queries, and Machine Learning programs which require a main memory workspace (that MapReduce does not offer)

Section 3

MapReduce

Maps and reductions

- **Map** and **reduction** are operations traditionally studied in functional programming
 - For example, reduction, a. k. a. fold, dates back to the APL language (1962)
- Recently they have formed the basis of **MapReduce**, a programming model for batch parallel processing of large datasets: the Map and Reduce operations are executed in parallel on the distributed dataset, producing a transformed dataset on disk
- Map and reduce functions are available now in many programming languages. Cf. Python's `map()` built-in function, and `reduce()` from the `functools` package

RDBMS or MapReduce?

- Relational Data Base Management Systems (RDBMS) are capable of performing distributed computation, for example in the form of distributed query answering
- Note however that RDBMS technology is optimised for short accesses to small amounts of data
 - The main structure is the index, which allows efficient access to the disk locations which contain the tuples having a given value, or range of values, in a column
- Sequential access has improved faster than random access. Thus, in state-of-the-art disk technology MapReduce is faster than a RDBMS for whole dataset reads and writes

RDBMS vs. MapReduce

	RDBMS	MapReduce
Size	GB	PB
Access	Interactive, batch	Batch
Updates	Multiple R/W	W once, R many
Transactions	ACID	No management
Unstructured data	N	Y
Integrity	Y	N
Scaling	N	Y

RDBMS and MapReduce complement each other, being different in all coordinates

White, T. (2009). Hadoop: The definitive guide 4th Edition. Reilly Media.

HPC vs. MapReduce: HPC

High-Performance Computing (HPC) focuses on distributing the computation across a cluster of computers for CPU-bound jobs, such as simulations, large linear-algebra computations

- Programmers explicitly control the subdivision of data and work into non-overlapping chunks which are processed by distinct computers of the cluster
- Coordination is also explicitly controlled by message-passing primitives, such as the Message Passing Interface (MPI)
- Failures during the computation must be handled in the program

HPC vs. MapReduce: MapReduce

In contrast, MapReduce in Hadoop focuses on data-intensive jobs, such as complex transformations and aggregations

- No explicit data and work partitioning: data are local to the computer which will process them
- No message passing: coordination is handled automatically in the MapReduce programming model
- Node failures are automatically handled

MapReduce

- $\text{map}(f, A)$ applies a function f to a collection of objects A . It corresponds to $\{f(x) : x \in A\}$
- $\text{reduce}(f, A)$ applies a function f of two arguments to a sequence of objects, accumulating intermediate results until the sequence is reduced to a single value. It corresponds to $f(\dots f(f(a_1, a_2), a_3) \dots a_n)$, with $A = (a_1, a_2, \dots, a_n)$
- In MapReduce, a computation is a sequence of alternating map and reduce function calls which operate in parallel on a distributed collection of data

The MapReduce programming model

- A computation receives a set of key/value pairs as input produces a set of key/value pairs as the output
- The programmer must specify a map and a reduce function
- Map produces a set of intermediate key/value pairs from an input key/value pair
- The intermediate pairs are grouped, or shuffled, by the values of the intermediate key and passed to the reduce function
- The reduce function merges the values for an intermediate key into a possibly smaller set of values

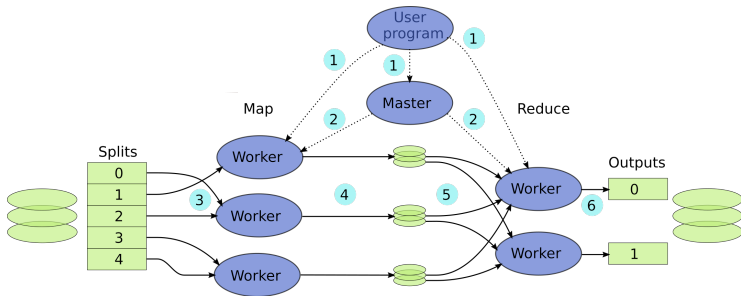
Google's implementation: Map

- The dataset is partitioned into a configurable number of **splits**, and copies of the program are executed on the nodes of the cluster
- One distinguished copy, the **master**, assigns map and reduce tasks to the **worker** copies when they are idle
- Each map worker reads a split, identifies input key-value pairs, and passes them to the user map function; the resulting intermediate key-value pairs are stored in main memory
- Intermediate pairs are written to the local disk from main memory at intervals, partitioned into labeled **regions**, so that a key is contained in regions with a unique label throughout the cluster. The locations of the regions are sent to the master, which communicates them to the reduce workers

Google's implementation: Reduce

- Each reduce worker collects the intermediate key-value pairs of its assigned regions from, possibly multiple, mappers' disks; at completion, the reduce worker sorts the pairs by key
- Each reduce worker scans its key-sorted pairs, collecting the values corresponding to the same key; when the pairs for that key are exhausted, it sends the collected values to the user reduce function, and proceeds with the next key. The function processes the key and values and writes the results to disk
- After the last task has terminated, the master passes control to the user program.

Google's MapReduce diagram



Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. Communications of the ACM, 51(1), 107–113. <https://doi.org/10.1145/1327452.1327492>

MapReduce Hadoop's Java API

- A mapper public class

```
import org.apache.hadoop.mapreduce.Mapper;  
public class MyMapperClass extends Mapper<KEYIN,VALUEIN,KEYOUT,VALUEOUT> {  
    // declare local objects here  
    public void map(KEYIN key, VALUEIN value, Context context) {  
        // body  
    }  
}
```

- A reducer public class

```
import org.apache.hadoop.mapreduce.Reducer;  
public class MyReducerClass extends Reducer<KEYIN,VALUEIN,KEYOUT,VALUEOUT> {  
    // declare local objects here  
    public void reduce(KEYIN key, Iterable<VALUEIN> value, Context context) {  
        // body  
    }  
}
```

MapReduce Java application outline

```
// Some imports, notably:
import org.apache.hadoop.mapreduce.Job;
public class MyMapReduceApp {
    public static void main(String[] args) {
        Job job = new Job();
        // set initialisations and file paths here
        job.setMapperClass(MaxTemperatureMapper.class);
        job.setReducerClass(MaxTemperatureReducer.class);
        job.setOutputKeyClass(KEYOUT.class);
        job.setOutputValueClass(VALUEOUT.class);
    }
}
```

The word count example for MapReduce

Count the occurrences of words in a text

MapReduce word count algorithm

- The map function generates intermediate key-value pairs of the form $(w, 1)$, where w is a word in the document, in parallel on all computing nodes
- The intermediate pairs are grouped by key in parallel on all nodes
- The reduce function sums all the unit integers associated to each w

Word count mapper in Python

```
#!/usr/bin/env python3
import sys
for line in sys.stdin:    # Read each line from stdin
    words = line.split()  # Get the words in each line
    for word in words:    # Generate the count for each word
        # Write the key-value pair to stdout to be processed by
        # the reducer.
        # The key is anything before the first tab character and the
        # value is anything after the first tab character.
    print('{0}\t{1}'.format(word, 1))
```

Word count reducer in Python

```
#!/usr/bin/env python3
import sys
curr_word = None
curr_count = 0
for line in sys.stdin: # Process key-value pairs from the mapper
    word, count = line.split('\t') # Key-value from the current line
    count = int(count) # Convert the count to an int
    # If the current word is the same as the previous word,
    # increment its count, otherwise print the word count
    if word == curr_word:
        curr_count += count
    else: # Write word and its number of occurrences
        # as a key-value pair to stdout
        if curr_word:
            print('{0}\t{1}'.format(curr_word, curr_count))
        curr_word = word
        curr_count = count
if curr_word == word: # Output the count for the last word
    print('{0}\t{1}'.format(curr_word, curr_count))
```

Run script

Scripts `mapper.py` and `reducer.py` can be first tested in Python, using pipes and sorting. Pipes connect the standard output of a process to the standard input of another, effectively providing a way of building simple processing pipelines at the operating system's shell level. In Windows' `cmd`

```
type chaucer.txt | python mapper.py | sort | python reducer.py
```

In Hadoop's MapReduce, the `run` script executes `hadoop`, passing the streaming jar file with the necessary options

```
#!/usr/bin/env bash
$HADOOP_INSTALL/bin/hadoop jar \
$HADOOP_INSTALL/share/hadoop/tools/lib/hadoop-streaming-3.3.0.jar \
-files ./mapper.py,./reducer.py \
-mapper ./mapper.py \
-reducer ./reducer.py \
-input ./chaucer.txt -output ./chaucer
```

The weather dataset example

- Sensors are among the most frequent sources of big data
- A prominent example is US climatic data, which are collected and made public by the National Climatic Data Center (NCDC) of the USA
- Collections are composed by plain ASCII-encoded text files, in a semi-structured format, with optional parts
- Excerpt from the Halifax Intl Airport 2020 file:

Station Id	Date	Time	Lat	Long	Elevation	Wind direction
0314713950999999	20200101	000004	+44883	-063500	FM-12+0145	999999V020120
0204713950999999	20200101	000004	+44881	-063509	FM-15+0145	999999V020130
0204713950999999	20200101	00214	+44881	-063509	FM-16+0145	999999V020120
0197713950999999	20200101	00374	+44881	-063509	FM-16+0145	999999V020120
0197713950999999	20200101	01004	+44881	-063509	FM-15+0145	999999V020120
0196713950999999	20200101	01304	+44881	-063509	FM-16+0145	999999V020140

- We will find the maximum temperature by year using MapReduce, which is a simple group by/max problem

Maximum temperature: Map

- Lines are scanned and the map function receives a sequence of key-value pairs
 - The byte offset of the first char of a line
 - The text line

For example, from the Halifax Intl Airport 2020 file:

```
(0, 0314713950999992020010100004+44883-063500FM...+0013... )  
(419, 0204713950999992020010100004+44881-063509FM...+0010... )  
(728, 0204713950999992020010100214+44881-063509FM...+0020... )
```

- The map function discards all chars except the year and the temperature and outputs them in pairs
 - (2020, +0013)
 - (2020, +0010)
 - (2020, +0020)

The weather dataset mapper

```
#!/usr/bin/env python3
import re
import sys
for line in sys.stdin:
    val = line.strip()
    (year, temp, q) = (val[15:19], val[87:92], val[92:93])
    if (temp != "+9999" and re.match("[01459]", q)):
        print("%s\t%s" % (year, temp))
```

Maximum temperature: Reduce

- Before the reduce step, the pairs are sorted by the framework
- The reduce function therefore receives

(2018, -0128)

(2018, -0130)

(2019, -0068)

(2019, -0070)

(2020, +0013)

(2020, +0010)

The weather dataset reducer

Iterate, accumulate the maximum if the key does not change, otherwise initialise the next maximum with the current value

```
#!/usr/bin/env python3
import sys
(last_key, max_val) = (None, -sys.maxsize)
for line in sys.stdin:
    (key, val) = line.strip().split("\t")
    if last_key and last_key != key:
        print("%s\t%s" % (last_key, max_val))
        (last_key, max_val) = (key, int(val))
    else:
        (last_key, max_val) = (key, max(max_val, int(val)))
if last_key:
    print("%s\t%s" % (last_key, max_val))
```

Run script

We concatenate the per-year data files in Datasets using the OS shell

```
cat Datasets/*-2018 Datasets/*-2019 Datasets/*-2020 > Datasets/ncdc
```

The run shell script calls hadoop passing the streaming jar file with the necessary options

```
#!/usr/bin/env bash
$HADOOP_INSTALL/bin/hadoop jar \
$HADOOP_INSTALL/share/hadoop/tools/lib/hadoop-streaming-3.3.0.jar \
-files ./mapper.py,./reducer.py \
-mapper ./mapper.py \
-reducer ./reducer.py \
-input ./Datasets/ncdc -output ./weather
```

Hadoop Distributed File System

HDFS was designed to achieve the following goals

- **Resilience**: automatic recovery from hardware failures, likely to occur in large computer clusters
- **Streaming access**: High throughput rather than low latency, because applications are batch, not interactive
- **Large data sets**: GB ÷ TB dataset size, high bandwidth, $\sim 100 \div 1000$ cluster nodes, $\sim 10^6 \div 10^7$ files
- **Write-once, read many**: Files are written, and once closed, they can be truncated, or data can be appended to them
- **Local computation**: Support for moving application close to the data
- **Portability**: Can be used on different platforms

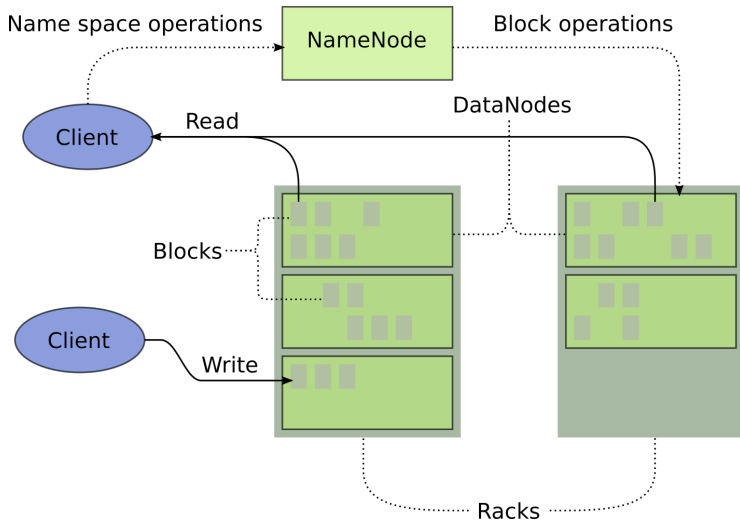
Apache Hadoop 3.3.4 – HDFS Architecture.

<https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>

HDFS architecture

- HDFS adopts a master/slave architecture
- One **NameNode**, the master of the architecture
 - Manages the name space of the system
 - Regulates file access by clients
- Multiple **DataNodes**, the slaves of the architecture
 - Associated 1-1 to cluster nodes
 - Manage the storage of the nodes they run on
- Data are contained in files. A file is split into blocks, which can be stored on multiple DataNodes
- Responsibilities of the NameNode
 - Association of blocks to DataNodes
 - File name space operations: open, close, rename
- Responsibilities of the DataNodes
 - Block management: create, delete, replicate
 - Read and write requests from clients

HDFS architecture diagram



HDFS command structure

- In the install directory of Hadoop, the `bin` directory contains programs and scripts
- Normally, the environment variable `HADOOP_INSTALL` is set to Hadoop's base directory
- We will assume the current directory's name is the value of `HADOOP_INSTALL`
- `bin/hdfs` is the script which invokes **HDFS commands**, as

```
bin/hdfs [SHELL_OPTIONS] COMMAND
        [GENERIC_OPTIONS] [COMMAND_OPTIONS]
```
- `SHELL_OPTIONS` are options valid for all Hadoop commands
- `GENERIC_OPTIONS` are supported by all commands
- `COMMAND_OPTIONS` are options specific to `COMMAND`

hdfs dfs commands (1)

- `bin/hdfs dfs` introduces a large family of commands, most of which are similar to commands of the Linux shell. The form is

```
bin/hdfs dfs [COMMAND [COMMAND_OPTIONS]]
```

where `COMMAND` is written with a leading dash

- List the content of the root directory of the HDFS distributed file system

```
bin/hdfs dfs -ls /
```

- Show the contents of a file

```
bin/hdfs dfs -cat /chaucer.txt
```

- Copy a file from the local file system to HDFS

```
bin/hdfs dfs -put ~/chaucer.txt /chaucer.txt
```

hdfs dfs commands (2)

- Copy the contents of a file in HDFS to the local file system

```
bin/hdfs dfs -get /chaucer.txt ~
```

- Create a directory

```
bin/hdfs dfs -mkdir /input
```

- Move a file or directory

```
bin/hdfs dfs -mv /chaucer.txt /input
```

- Copy a file

```
bin/hdfs dfs -cp /input/chaucer.txt /chaucer.txt
```

- Remove a file

```
bin/hdfs dfs -rm /chaucer.txt
```

Cf. [Apache Hadoop 3.3.4 – Overview](#).

Pseudo-distributed configuration files

- Hadoop's configuration files reside in `etc/hadoop`
- The default configuration of Hadoop is a **standalone** configuration, i.e., non-distributed
- The minimal configuration allowing for the employment of HDFS is **Pseudo-Distributed** mode, which requires a modification of `core-site.xml` and `hdfs-site.xml`

`core-site.xml`

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

`hdfs-site.xml`

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
</configuration>
```

Pseudo-distributed startup

- Switch to the user the Hadoop system was installed with

```
sudo -i -u ubuntu bash
```

- Test secure shell connection to localhost. The HDFS startup script relies on it. See the [shell instructions](#) on Hadoop's site to set up localhost ssh access

```
ssh localhost  
exit
```

- The distributed file system must be formatted

```
bin/hdfs namenode -format
```

- Hadoop requires a NameNode daemon and a DataNode daemon listening for connections. The sbin directory contains utility scripts to start and stop Hadoop components.

```
sbin/start-dfs.sh
```

Namenode web interface

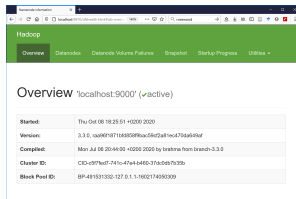
Script `sbin/start-dfs.sh` sets up a web server listening on TCP port 9870
If Hadoop runs on a local Vagrant virtual machine without a web client

`vagrant ssh -- -L 9870:localhost:9870`

creates a tunnel from local port 9870 to the virtual machine port 9870

- Note. `localhost` is the conventional name of the host `ssh` is logging into. Specifying another host would set up a tunnel to the login host, and a non-encrypted channel to that host.

- The Namenode information page can be accessed from the host system's web browser at address `http://localhost:9870`



Word count in pseudo-distributed mode

- In pseudo-distributed and fully-distributed mode, input files must be read from the distributed file system, not from the OS file system
- Write `chaucer.txt` to HDFS, for example to the root. (Do that as user `ubuntu`.)

```
$HADOOP_INSTALL/bin/hdfs dfs -put \  
/vagrant/Hadoop/WordCount/chaucer.txt /
```

- Run script

```
#!/usr/bin/env bash  
$HADOOP_INSTALL/bin/hdfs dfs -rm -f -r /chaucer #dfs command  
$HADOOP_INSTALL/bin/hadoop jar \  
$HADOOP_INSTALL/share/hadoop/tools/lib/hadoop-streaming-3.3.0.jar \  
-files ./mapper.py,./reducer.py \  
-mapper ./mapper.py \  
-reducer ./reducer.py \  
-input /chaucer.txt -output /chaucer
```

Weather in pseudo-distributed mode

Let's exploit user directories in HDFS. They are of the form
/user/<username>:

```
$HADOOP_INSTALL/bin/hdfs dfs -mkdir /user  
$HADOOP_INSTALL/bin/hdfs dfs -mkdir /user/ubuntu
```

Copy the NCDC weather file to the user directory

```
$HADOOP_INSTALL/bin/hdfs dfs -put Datasets/ncdc
```

```
#!/usr/bin/env bash
```

```
$HADOOP_INSTALL/bin/hadoop jar \  
$HADOOP_INSTALL/share/hadoop/tools/lib/hadoop-streaming-3.3.0.jar \  
-files ./mapper.py,./reducer.py \  
-mapper ./mapper.py \  
-reducer ./reducer.py \  
-input ncdc -output weather
```

Min, max and nearest neighbours

- Solutions to minimum and maximum problems can be expressed as repeated applications of associative and commutative functions $\min(a, b)$ and $\max(a, b)$
- Solving the (1-) nearest neighbour problem, given a p -dimensional point x and data set, amounts to solving a minimum problem on the set of distances between x and the data set elements, and returning a data set element having the minimum distance from x , and its distance from x

Nearest neighbours map and reduce

- The associative and commutative function which is the basis of the reduce step is then

$$((x_1, d_1), (x_2, d_2)) \rightarrow \begin{cases} (x_1, d_1), & \text{if } d_1 < d_2; \\ (\text{tiebreaker}(x_1, x_2), d_1), & \text{if } d_1 = d_2; \\ (x_2, d_2), & \text{otherwise.} \end{cases}$$

where $d_1 = d(x, x_1)$, $d_2 = d(x, x_2)$, d is the chosen distance function, x is the point the nearest neighbour of which must be computed, and tiebreaker is any deterministic choice of one of two points

- The map step has responsibility for preparing a set of pairs $(x_i, d(x, x_i))$, for $i = 1, 2, \dots$

Nearest neighbour mapper

Example with $p = 2$ and $x = (0, 0)$

```
#!/usr/bin/env python3
import sys

def sq_sum(x):
    return sum((y ** 2 for y in x))

def sq_dist(x, y):
    return sq_sum((z[0] - z[1] for z in zip(x, y)))

for line in sys.stdin:
    id_coords = line.split(',') # CSV file
    id = int(id_coords[0]) # 1st field: point id
    # coordinates in the remaining fields
    coords = [float(c) for c in id_coords[1:]]
    # Generate the distance value
    sd = sq_dist(coords, (0, 0))
    print('{0}\t{1}'.format(id, sd))
```

Nearest neighbour reducer

```
#!/usr/bin/env python3
import sys
import math
(last_id, min_sd) = (None, sys.maxsize)
# Process each key-value pair from the mapper
for line in sys.stdin:
    # Get the id and squared distance from the current line
    (id, sd) = line.split('\t')
    sd = float(sd) # Convert the distance to a float
    if sd < min_sd:
        (last_id, min_sd) = (id, sd)
    elif sd == min_sd:
        (last_id, min_sd) = (min(last_id, id), min_sd)
print("%s\t%s" % (last_id, math.sqrt(min_sd)))
```

Nearest neighbour set mapper 1

```
#!/usr/bin/env python3
import sys
import csv

def sq_sum(x):
    return sum((y ** 2 for y in x))

def sq_dist(x, y):
    return sq_sum((z[0] - z[1] for z in zip(x, y)))

query_file = str(sys.argv[1]) # name of query file
with open(query_file, newline='') as csvfile:
    reader = csv.reader(csvfile, delimiter=',')
    queries = [] # fill list with points in file
    for q in reader:
        queries.append([float(x) for x in q])
```

Nearest neighbour set mapper 2

```
for line in sys.stdin:
    id_coords = line.split(',')
    id = int(id_coords[0])
    coords = [float(c) for c in id_coords[1:]]
    # Generate distance values for every query point
    q_id = 1
    for q in queries:
        sd = sq_dist(coords, q)
        print('{0}\t{1}\t{2}'.format(q_id, id, sd))
        q_id += 1
```

Nearest neighbour set reducer

```
#!/usr/bin/env python3
import sys
import math
(last_q_id, id_min, min_sd) = (None, None, sys.maxsize)
for line in sys.stdin:
    # Get the id and squared distance from the current line
    (q_id, id, sd) = line.split('\t')
    sd = float(sd) # Convert the distance to a float
    if last_q_id and last_q_id != q_id:
        print("%s\t%s\t%s" % (last_q_id, id_min, math.sqrt(min_sd)))
        (last_q_id, id_min, min_sd) = (q_id, id, sd)
    else:
        if sd < min_sd:
            (last_q_id, id_min, min_sd) = (q_id, id, sd)
        elif sd == min_sd:
            (last_q_id, id_min, min_sd) = (q_id, min(id_min, id), min_sd)
print("%s\t%s\t%s" % (last_q_id, id_min, math.sqrt(min_sd)))
```

Fully distributed configuration (cluster)

- In `core-site.xml`, replace `localhost` with the host name of the node manager host, e. g., `hdmaster`
- `hdfs-site.xml`
 - requires the `dfs.replication` property set. Having more than one data node, it can be set to a value > 1 . Moreover, we must set:
 - In the `NameNode`, the `dfs.namenode.name.dir` property. The namespace and logs are stored here
 - In the `DataNode`, the `dfs.datanode.data.dir` property. File blocks are stored here

`core-site.xml`

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://hdmaster:9000</value>
  </property>
</configuration>
```

Fully distributed hdfs-site.xml

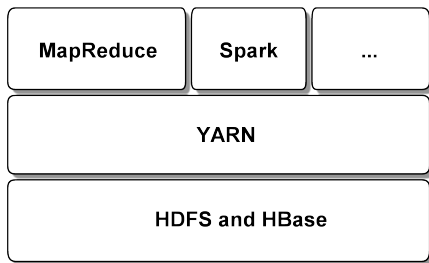
Set replication ≥ 2 , and define work directories for the name node and the datanodes

hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>2</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:/home/ubuntu/hdfs/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:/home/ubuntu/hdfs/datanode</value>
  </property>
</configuration>
```

Resource management in Hadoop

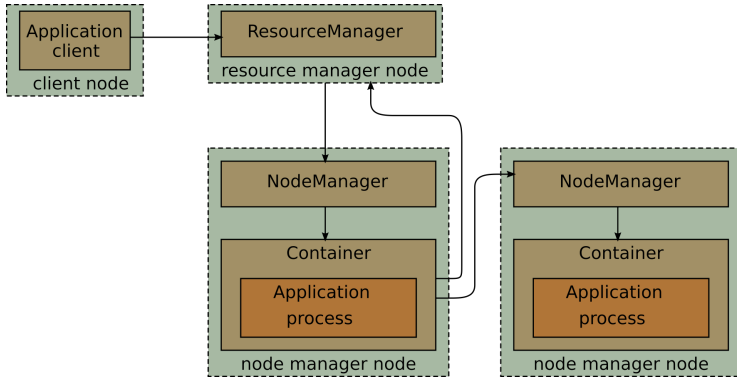
- Starting with Hadoop 2, a resource manager named YARN (Yet Another Resource Manager) was added
- Resource management is not normally under user control, although YARN provides an **application programming interface** (API) which exposes cluster resources



YARN

- Operates through two daemons, the **resource manager** and the **node manager**
- There is one resource manager per cluster, whereas node managers run on every node
- It's the responsibility of a node manager to create and manage **containers**, which are processes with a bounded amount of each computational resource such as CPU and memory
- The process (the application client) initially sends a request to the resource manager to start an **application master** as a first step of a **YARN application**
- To fulfil the request, the resource manager contacts a node manager and asks it to run a container and the application master as a process inside it

YARN application run



White, T. (2009). Hadoop: The definitive guide 4th Edition. Reilly Media.

Application master

- The application master algorithm is application-dependent
- In its simplest form, it performs a computation using the container's allocated resources and returns
- Instead, a MapReduce application sends requests to the resource manager to obtain other containers to run a distributed computation
- Requests for containers can specify both the amount of resources for the container, and locality constraints, such as a specific node. Because the latter constraint cannot always be satisfied, YARN tries to find the closest node to the requested one
- The flexibility of YARN requests allows for the computation to be performed as close to the data as possible

YARN configuration

- Modify `etc/hadoop/yarn-site.xml`
- Set the hostname of the resource manager host at node manager nodes. For example, if the resource manager is `hdresource`

```
<property>  
  <name>yarn.resourcemanager.hostname</name>  
  <value>hdresource</value>  
</property>
```

- Set the amount of physical memory in MB the container is allowed to use with `yarn.nodemanager.resource.memory-mb`

```
<property>  
  <name>yarn.nodemanager.resource.memory-mb</name>  
  <value>256</value>  
</property>
```

YARN configuration (cont'd)

- At node manager nodes, the shuffle service for Map Reduce with property `yarn.nodemanager.aux-services`

```
<property>  
  <name>yarn.nodemanager.aux-services</name>  
  <value>mapreduce_shuffle</value>  
</property>
```

- At both the resource manager nodes and node manager nodes, to enable log aggregation

```
<property>  
  <name>yarn.log-aggregation-enable</name>  
  <value>true</value>  
</property>
```

MapReduce application configuration

etc/hadoop/mapred-site.xml allows to configure the size of many resources available to the MapReduce job, such as heap sizes and memory allocation limits

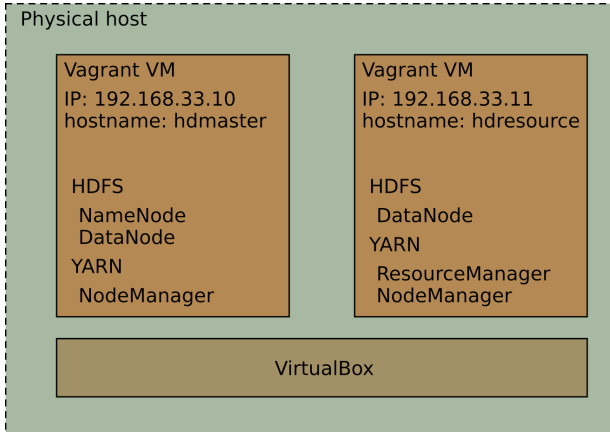
- A full list is available at [MapReduce](#)
- A basic distributed configuration needs the execution framework, `mapreduce.framework.name`, property set to `yarn`

mapred-site.xml

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

Cluster setup with vagrant VMs

- Set up of two vagrant VMs, with distinct hostnames and IP numbers



Cluster setup with vagrant VMs (cont'd)

Configure in Vagrantfile

- Virtual local private network among the physical host and the virtual machines with Internet Protocol number
- Increment virtual RAM size
- Set the bootstrap file name
- Hostname (writes the /etc/hostname file in the VM)

Vagrantfile

```
config.vm.network "private_network", ip: "192.168.33.10"
config.vm.provider "virtualbox" do |vb|
  vb.memory = "4096"
end
config.vm.provision :shell, path: "bootstrap.sh"
config.vm.hostname = "hdmaster" # or hdmaster
```

Cluster setup: Hosts file and startup

- Host names are referenced in the configuration files of Hadoop
- Ensure /etc/hosts contains the mapping between IP numbers and host names of hdmaster and hdresource

hdmaster:/etc/hosts

```
192.168.33.11  hdresource
192.168.33.10  hdmaster  hdmaster
```

hdresource:/etc/hosts

```
192.168.33.10  hdmaster
192.168.33.11  hdresource  hdresource
```

Cluster startup

- Start the name node and the data nodes

```
bin/hdfs --daemon start namenode
```

```
bin/hdfs --daemon start datanode # on all nodes
```

- Start the resource manager, node managers, and the proxy server

```
bin/yarn --daemon start resourcemanager
```

```
bin/yarn --daemon start nodemanager # on all nodes
```

```
bin/yarn --daemon start proxyserver
```

- Start the history server

```
bin/mapred --daemon start historyserver
```

Cf. the [Hadoop cluster guide](#)

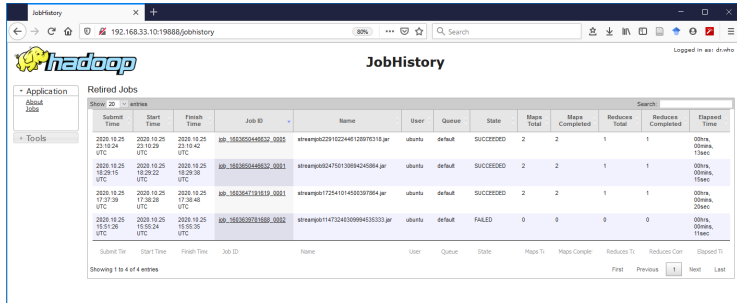
Monitoring status

- Web servers are open on designated TCP ports
- To access the web pages, use the host names of the name node, data nodes, and history server node, if they are known to the host running the web browser; use IP numbers otherwise
- For example, in the Vagrant VM setup, on the physical host's web browser
 - Name node `http://192.168.33.10:9870`
 - Data nodes `http://192.168.33.10:9864` `http://192.168.33.11:9864`
 - History server `http://192.168.33.10:19888` (if the history server runs on `hdmaster`)

IP addresses have been used in this case because the `hdmaster` and `hdresource` host names had been defined inside the local `/etc/hosts` files in the VMs; thus the name-to-IP address translation for those host names is available only to the operating systems hosting those `/etc/hosts` files, and not to the physical host

Running the application

- Scripts for the pseudo-distributed case work for the fully-distributed case as well with no changes
- Finished MapReduce jobs are listed in the history server web page



The screenshot shows the Hadoop JobHistory web interface. The browser address bar displays the URL 192.168.33.10:19888/jobhistory. The page title is "JobHistory" and it shows "Logged in as: drwho". On the left, there is a sidebar with "Application" (About Jobs), "Tools", and "Retired Jobs". The main content area displays a table of retired jobs. The table has columns for Submit Time, Start Time, Finish Time, Job ID, Name, User, Queue, State, Maps Total, Maps Completed, Reduces Total, Reduces Completed, and Elapsed Time. There are four entries in the table, all with a state of "SUCCEEDED" except for the last one which is "FAILED".

Submit Time	Start Time	Finish Time	Job ID	Name	User	Queue	State	Maps Total	Maps Completed	Reduces Total	Reduces Completed	Elapsed Time
2020-10-25 23:10:24 UTC	2020-10-25 23:10:29 UTC	2020-10-25 23:10:42 UTC	job_16836504458632_0000	streamjob2291022448126076318.jar	ubuntu	default	SUCCEEDED	2	2	1	1	00hrs, 00mins, 13sec
2020-10-25 18:29:15 UTC	2020-10-25 18:29:22 UTC	2020-10-25 18:29:38 UTC	job_16836504458632_0001	streamjob6524750130684245884.jar	ubuntu	default	SUCCEEDED	2	2	1	1	00hrs, 00mins, 15sec
2020-10-25 17:37:39 UTC	2020-10-25 17:38:28 UTC	2020-10-25 17:38:48 UTC	job_1683647191619_0001	streamjob172541014500397984.jar	ubuntu	default	SUCCEEDED	2	2	1	1	00hrs, 00mins, 20sec
2020-10-25 15:51:25 UTC	2020-10-25 15:55:24 UTC	2020-10-25 15:55:35 UTC	job_1683639781668_0002	streamjob1147324030999453533.jar	ubuntu	default	FAILED	0	0	0	0	00hrs, 00mins, 11sec

Showing 1 to 4 of 4 entries

First Previous 1 Next Last

Section 4

Spark

Limitations of MapReduce

While MapReduce has proven to be very effective in processing large datasets in data-intensive applications, it assumes that processing can be expressed as an acyclic flow of data through a series of operations.

However, many data processing algorithms violate such an assumption by keeping working sets between two or more iterations. Notably, many algorithms in machine learning are iterative and require that values are preserved between consecutive iterations.

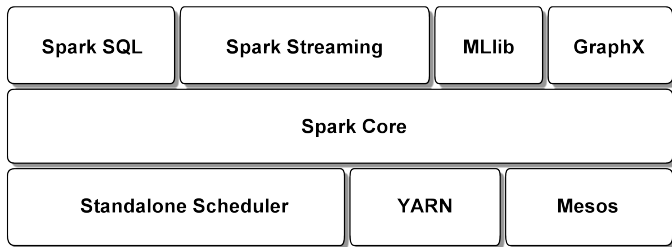
- In MapReduce, results must be stored on disk at the end of a job
- Thus, each iteration would be implemented as a MapReduce job, and each job would store its result on disk for the next job
- Reading and writing to disk are slower than readings and write to RAM, compare DDR4 and SSD SATA 6GB / s or SAS

The Spark project

Cluster computing platform designed for speed and polyvalency

- In memory computation
- More efficient than MapReduce, also for disk computations
- Workloads: batch applications, iterative algorithms, interactive queries, streaming applications
- Application Programming Interface (API): Java, Python, R, Scala
- Spark is an open-source project that originates from UC Berkeley in 2009
 - Switch to the Apache Software Foundation in 2013

Architecture of Spark



- Spark Core: Task, memory, fault recovery, storage
- Spark SQL: SQL and HQL interface
- Spark Streaming: live streams of data
- MLlib: Machine learning library
- GraphX: Graph manipulation, parallel computations
- Cluster Managers: Standalone (internal), YARN, Mesos

Spark processing modes

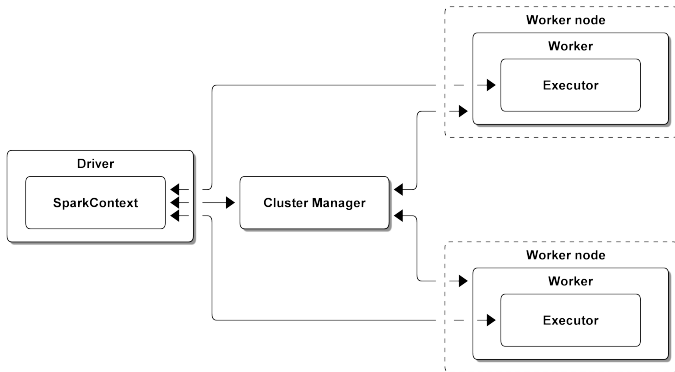
A **Spark application** consists of a **driver program** that coordinates a set of **executor** processes

- When the driver and the executors are run on different nodes of a cluster, Spark is running in **cluster mode**
 - Spark uses a cluster manager to be informed of the availability of resources
 - This mode is intended for production
- When the driver and the executors are run on the same machine as different processes, Spark is running in **local mode**
 - This mode is mostly used for exploring data, experimenting, programming, and debugging

Driver, worker, executor

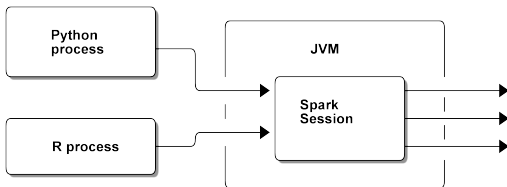
- Driver
 - Contains the main function
 - Defines distributed data sets
 - Performs operations on datasets
- Worker
 - Process that remains in execution and keeps the datasets distributed in RAM
 - Normally only one for each worker node
- Executor
 - Performs **tasks**
 - Dialogue with a single application
 - Running for the duration of the application
 - Possibly multiple executors for each worker

Spark distributed architecture



Spark languages

- Language Application Programmer Interfaces (APIs) are available for the following languages
 - **Scala**. The language Spark is written in. The Scala programs are compiled to JVM bytecode
 - **Java**
 - **Python** Exposes almost all functionalities available in Scala
 - **R**. Through either **SparkR** or **sparklyr**
 - **SQL**. Supports the ANSI 2003 standard



Starting Spark

The \$SPARK_INSTALL/bin directory contains scripts

- to run Spark interactively, using the language APIs

Scala spark-shell

Python pyspark

SQL spark-sql

- to submit a batch job to Spark
 - spark-submit

Low level / High level APIs

Spark's own APIs, which are the basis for the functionalities the various language APIs provide, are of two kinds

- **High level APIs** (Structured APIs)
 - Datasets
 - DataFrames
 - SQL tables
- **Low level APIs** (Unstructured APIs)
 - Resilient Distributed Datasets (RDD)
 - Shared variables

Section 5

Spark structured API

DataFrames, Datasets

- Both are data collections which can be represented as a table, in a sense of the term similar to that in SQL
 - A schema is defined for the collection, with column names and column types. The schema can be defined using an instruction, or automatically by Spark when data are read from the source
 - Values of a column have the type of the column
 - All columns have the same length
- DataFrames are available in all languages, whereas Datasets are available only to languages which use the JVM (Scala and Java)
- Conformance of values to types of columns is performed at runtime for DataFrames, at compile time for Datasets

DataFrame creation

```
source = "Spark/Data/measurements.json"
```

The JSON file above contains rows like

```
{ "Code": 88118, "Date": "2015-10-30", "Pollutant": "Ozone", ... },  
{ "Code": 927170, "Date": "2015-10-08", "Pollutant": "Particulate matter", ... },
```

Create a DataFrame from it

```
df = spark.read.format("json").load(source)  
df.show()
```

Code	Concentration	Date	Pollutant	Station
88118	79	2015-10-30	Ozone	Rimini
927170	25	2015-10-08	Particulate matter	Ravenna
190115	59	2015-02-04	Ozone	Rimini

Note We assume pyspark has been launched in the directory containing the Spark directory

DataFrame schema-on-read

The schema-on-read feature of Spark infers the following schema

```
df.schema
```

```
StructType(List(StructField(Code,LongType,true),  
  StructField(Concentration,LongType,true),  
  StructField(Date,StringType,true),  
  StructField(Pollutant,StringType,true),  
  StructField(Station,StringType,true)))
```

```
df.printSchema()
```

```
root
```

```
|-- Code: long (nullable = true)  
|-- Concentration: long (nullable = true)  
|-- Date: string (nullable = true)  
|-- Pollutant: string (nullable = true)  
|-- Station: string (nullable = true)
```

Columns

- Column manipulation requires importing `col` or `column` from `pyspark.sql.functions`

```
from pyspark.sql.functions import col
```

```
from pyspark.sql.functions import column
```

- A reference to a column is

```
col("Pollutant")    # or
```

```
column("Pollutant")
```

- Note that such references do not correspond to values outside a `DataFrame`

Expressions

- An expression in Spark is a set of transformations on values of a data frame row
- As in SQL, expressions are based on constants, operators, columns
- They require importing `expr` from `pyspark.sql.functions`
- The resulting underlying computation of an expression will be exactly the same of a semantically equivalent `DataFrame` expression

```
from pyspark.sql.functions import expr
expr("Concentration < 50 and Concentration > 10") # SQL-like
(col("Concentration")) < 50 & (col("Concentration") > 10) # DataFrame
expr("Concentration * .9 + 0.01") # SQL-like
col("Concentration") *.9 + 0.01 # DataFrame
```

Rows

- Row manipulation requires importing Row from `pyspark.sql`
- A row is created with the Row function

```
from pyspark.sql import Row  
meas = Row(88118, "2015-10-30", "Ozone", "Rimini", 79)
```

Rows are accessible as Python slices in the Python API

```
meas[0]
```

```
meas[1:4]
```

```
88118
```

```
('2015-10-30', 'Ozone', 'Rimini')
```

Data frames from rows

```
from pyspark.sql import Row
from pyspark.sql.types import StructField, StructType, StringType, LongType
AirSchema = StructType([
    StructField("Code",StringType(),False),
    StructField("Date",StringType(),True),
    StructField("Pollutant",StringType(),True),
    StructField("Station",StringType(),False, metadata={"station": "name"}),
    StructField("Concentration",LongType(),True)
])
meas1 = Row(88118,"2015-10-30","Ozone","Rimini",79)
meas2 = Row(927170,"2015-10-08","Particulate matter","Ravenna",25)
measurements = spark.createDataFrame([meas1, meas2], AirSchema)
measurements.show()
```

Code	Date	Pollutant	Station	Concentration
88118	2015-10-30	Ozone	Rimini	79
927170	2015-10-08	Particulate matter	Ravenna	25

Column selection

The select method of DataFrame projects the frame onto specified column names, possibly renamed using an expression

```
df.select("Date", "Station", "Pollutant").show(2)
```

Date	Station	Pollutant
2015-10-30	Rimini	Ozone
2015-10-08	Ravenna	Particulate matter

Column selection (cont'd)

- Two ways of renaming an output column: an expression, or the `alias` method

```
df.select(expr("Date").alias("measure_date")).show(2)
```

```
df.select(expr("Date as measure_date")).show(2)
```

```
+-----+
|measure_date|
+-----+
|  2015-10-30|
|  2015-10-08|
|  2015-02-04|
|  2015-12-13|
+-----+
```

Selecting columns with expressions

The `selectExpr` method allows for adding computed columns to the output data frame using expressions

```
df.selectExpr("*",  
  "Concentration < 50 and Concentration > 10 as Interv").show(3)
```

Code	Concentration	Date	Pollutant	Station	Interv
88118	79	2015-10-30	Ozone	Rimini	false
927170	25	2015-10-08	Particulate matter	Ravenna	true
190115	59	2015-02-04	Ozone	Rimini	false

Aggregations in expressions

The correspondence with SQL extends to SQL statistical functions

```
df.selectExpr("avg(Concentration)",  
              "max(Concentration)",  
              "count(distinct(Code))").show()
```

avg(Concentration)	max(Concentration)	count(DISTINCT Code)
48.34	100	100

Literal values

Constant columns can be added to the select expressions using **literals**

```
from pyspark.sql.functions import lit
```

```
df.select("*", lit("valid").alias("Validity")).show(3)
```

Code	Concentration	Date	Pollutant	Station	Validity
88118	79	2015-10-30	Ozone	Rimini	valid
927170	25	2015-10-08	Particulate matter	Ravenna	valid
190115	59	2015-02-04	Ozone	Rimini	valid

Adding or replacing columns

The `withColumn` method of `DataFrame` adds a column or replaces it using a column expression as source for its values

```
df.withColumn('Square conc', df.Concentration**2).show(2)
```

Code	Concentration	Date	Pollutant	Station	Square conc
88118	79	2015-10-30	Ozone	Rimini	6241.0
927170	25	2015-10-08	Particulate matter	Ravenna	625.0

- With a literal, it adds a constant column

```
df.withColumn("Validity", lit("valid")).show(3)
```

Removing columns

- Removing, or dropping, of a column

```
df.withColumn('Square conc', df.Concentration**2)\  
  .drop("Concentration").show(2)
```

Code	Date	Pollutant	Station	Square conc
88118	2015-10-30	Ozone	Rimini	6241.0
927170	2015-10-08	Particulate matter	Ravenna	625.0
190115	2015-02-04	Ozone	Rimini	3481.0

Filtering

- where or filter with a string or a column expression
- Conjoined expressions: Chained where, or and

```
# could also use .filter instead of .where  
df.where(col("Concentration") > 90)\  
  .where(col("Date") ≥ '2015-10-16').show()
```

Code	Concentration	Date	Pollutant	Station
541554	95	2015-10-16	Nitrogen dioxide	Ravenna
720857	92	2015-11-25	Particulate matter	Modena

```
df.where("Concentration > 90 and Date ≥ '2015-10-16']").show()
```

Duplicate removal

Duplicate rows can be removed from the output with the `distinct()` method of DataFrame

```
df.select("Station", "Pollutant")\
  .distinct()\
  .sort("Station", "Pollutant")\
  .show()
```

Station	Pollutant
Bologna	Benzene
Bologna	Nitrogen dioxide
Bologna	Ozone
Bologna	Particulate matter
Cesena	Benzene
Cesena	Nitrogen dioxide
Cesena	Ozone
Cesena	Particulate matter
Modena	Benzene
Modena	Nitrogen dioxide
Modena	Ozone
Modena	Particulate matter
Ravenna	Benzene
Ravenna	Nitrogen dioxide
Ravenna	Ozone
Ravenna	Particulate matter
Rimini	Benzene
Rimini	Nitrogen dioxide
Rimini	Ozone
Rimini	Particulate matter

Sorting

- With the `sort()` or `OrderBy()` DataFrame methods

```
df.sort("Date").show(3)
```

Code	Concentration	Date	Pollutant	Station
23433	48	2015-01-06	Benzene	Ravenna
516267	19	2015-01-07	Particulate matter	Modena
738018	43	2015-01-17	Ozone	Rimini

```
df.orderBy("Pollutant", "Concentration").show(3)
```

Code	Concentration	Date	Pollutant	Station
853181	0	2015-01-31	Benzene	Cesena
225699	10	2015-06-22	Benzene	Ravenna
770774	14	2015-06-24	Benzene	Bologna

Ascending and descending order

- Using the `orderBy()` method with either the `asc()` or the `desc()` method of a column

```
from pyspark.sql.functions import desc, asc
df.orderBy(col("Pollutant"), col("Concentration")\
    .desc()).show(5)
```

Code	Concentration	Date	Pollutant	Station
403863	100	2015-07-25	Benzene	Ravenna
240104	93	2015-05-16	Benzene	Modena
100364	81	2015-03-20	Benzene	Modena
443801	80	2015-12-26	Benzene	Rimini
435727	79	2015-12-02	Benzene	Modena

Limit output rows

- Limit the number of output rows with `limit()`

```
top = df.orderBy(col("Concentration").desc()).limit(3)
top.show()
```

Code	Concentration	Date	Pollutant	Station
403863	100	2015-07-25	Benzene	Ravenna
792966	98	2015-07-31	Ozone	Bologna
104794	97	2015-03-10	Particulate matter	Bologna

Collecting

The `take()`, `show()`, and `collect()` methods of `DataFrame` collect a fraction of its rows

- Show the first 3

```
df.show(3)
```

- First 3 as list

```
first3 = df.take(3)
```

- All rows as list

```
all = df.collect()
```

Note Never use collecting methods carelessly. If the total size of the returned rows exceeds the amount of memory on the driver, the driver might crash

Joins

- In a relational DBMS, the **equi-join** of tables T_1 and T_2 on equality predicates $A_1 = B_1, \dots, A_k = B_k$, consists of every row r on all the columns of T_1 and T_2 such that its projection on the columns of T_i is in T_i ($i = 1, 2$) and $r.A_j = r.B_j, j = 1, 2, \dots, k$
- General joins evaluate a **join expression** involving the $r.A_j$'s and the $r.B_j$'s, instead of a conjunction of equalities
- Equi-joins are by far the most frequent kind of query in relational databases, because such databases consist of a multitude of tables in third normal form
- Likewise, in data analysis, models are constructed from several datasets, which need to be joined. However, equi-joins are not generally sufficient

Inner join

- Load a csv of vehicles by year and province

```
cities_source = "Spark/Data/cities.csv"
cities = spark.read.csv(cities_source, header=True,\
                        inferSchema=True)
```

- An **inner join** contains exactly the rows for which $r.A_j = r.B_j$,
 $j = 1, 2, \dots, k$

```
joinExpr = df["Station"] = cities["City"]
df.join(cities, joinExpr).show(4)
```

Code	Concentration	Date	Pollutant	Station	City	Area	Population
88118	79	2015-10-30	Ozone	Rimini	Rimini	135.71	151200
927170	25	2015-10-08	Particulate matter	Ravenna	Ravenna	241.75	158058
190115	59	2015-02-04	Ozone	Rimini	Rimini	135.71	151200
697379	0	2015-12-13	Ozone	Cesena	Cesena	249.47	97190

Outer join

Let J be the inner join of T_1 and T_2 , and S_1 and S_2 be their columns respectively. An **outer join** contains all the rows of J , and in addition any row r such that

- 1 either its projection on S_2 is null and its projection on S_1 is in T_1 , but not in the projection of J on S_1
- 2 or its projection on S_1 is null and its projection on S_2 is in T_2 , but not in the projection of J on S_2 ,

That is, for each row in either T_1 or T_2 for which no match could be found with the inner join, its “combination” with the null row on the columns of the other table is added to the output

```
df.join(cities, joinExpr, "outer").show(4)
```

Code	Concentration	Date	Pollutant	Station	City	Area	Population
88118	79	2015-10-30	Ozone	Rimini	Rimini	135.71	151200
...	...	...	...	...	...	...	...
null	null	null	null	null	Ferrara	405.16	132195

Left outer, right outer, left semi

- In a **left outer join** only rows satisfying 1. are added. Similarly, in a **right outer join** only rows satisfying 2. are added
 - Note that the DataFrame with the join method has the role of T_1
- A **left semi join** contains the rows of T_1 which satisfy the join expression

```
cities.join(df, joinExpr, "left_semi").show()
```

City	Area	Population
Rimini	135.71	151200
Cesena	249.47	97190
Bologna	140.86	391525
Ravenna	241.75	158058
Modena	183.19	186741

Left anti joins, cartesian products

- A **left anti join** contains the rows of T_1 which do not satisfy the join expression

```
cities.join(df, joinExpr, "left_anti").show()
```

City	Area	Population
Ferrara	405.16	132195

- A **cartesian** or **cross** product is equivalent to a join with a True join expression

```
cities.join(df, joinExpr, "cross").show(3)
```

City	Area	Population	Code	Concentration	Date	Pollutant	Station
Rimini	135.71	151200	88118	79	2015-10-30	Ozone	Rimini
Ravenna	241.75	158058	927170	25	2015-10-08	Particulate matter	Ravenna
...	...	...	...	...	...	...	...
Ferrara	405.16	132195	888924	45	2015-02-03	Benzene	Ravenna

Aggregations

- Counting

```
from pyspark.sql.functions import count, countDistinct  
df.select(count("Station")).show()  
df.select(countDistinct("Station")).show()
```

```
+-----+  
|count(Station)|  
+-----+  
|           100|  
+-----+
```

```
+-----+  
|count(DISTINCT Station)|  
+-----+  
|                5|  
+-----+
```

- Similarly, basic functions like first, last, min, max, sum, sumDistinct are computable by importing them from `pyspark.sql.functions`

```
from pyspark.sql.functions import sum  
cities.select(sum("Area")).show()
```

```
+-----+  
|sum(Area)|  
+-----+  
|  1356.14|  
+-----+
```

Variance, covariance, correlation

- Both sample and population

```
from pyspark.sql.functions import var_pop, var_samp,\
    corr, covar_pop, covar_samp
df.select(var_samp("Concentration"), var_pop("Concentration"))
```

var_samp(Concentration)	var_pop(Concentration)
903.3175757575759	894.2844000000001

```
conc_pop = df.join(cities, df["Station"] = cities["City"])
conc_dens = conc_pop.withColumn("Density",\
    conc_pop["Population"]/conc_pop["Area"])
conc_dens.select(corr("Density", "Concentration"),\
    covar_samp("Density", "Concentration"),\
    covar_pop("Density", "Concentration")).show()
```

corr(Density, Concentration)	covar_samp(Density, Concentration)	covar_pop(Density, Concentration)
0.05278344955861994	1413.1404856685933	1399.0090808119076

Collecting values

Aggregation as collection of values of columns into sets or lists:

```
from pyspark.sql.functions import collect_set, collect_list
```

```
df.agg(collect_set("Station"),\
       collect_list("Concentration")).show()
```

```
+-----+-----+
|collect_set(Station)|collect_list(Concentration)|
+-----+-----+
|[Bologna, Modena,...|      [79, 25, 59, 0, 5...|
+-----+-----+
```

Aggregations by groups

```
df.groupBy("Station").count().show()
```

```
+-----+-----+
|Station|count|
+-----+-----+
| Modena|   19|
| Bologna|  24|
| Cesena|   17|
| Rimini|   16|
| Ravenna|  24|
+-----+-----+
```

```
df.groupBy("Station", "Pollutant").agg(avg("Concentration").alias("Avg conc"),\
    var_samp("Concentration").alias("Var conc")).show(3)
```

```
+-----+-----+-----+-----+
|Station|Pollutant|Avg conc|Var conc|
+-----+-----+-----+-----+
|Ravenna|Nitrogen dioxide|59.5|1047.0|
|Bologna|Particulate matter|59.75|1022.25|
|Rimini|Ozone|60.42857142857143|514.6190476190476|
+-----+-----+-----+-----+
```

Functions on windows

Functions are applicable to sliding windows, defined by **window specifications**, which set

- **grouping**: `partitionBy` creates groups according to argument columns or expressions
- **ordering**: `orderBy` orders rows inside groups according to argument columns or expressions
- **framing**:
 - `rowsBetween` defines a window of rows, in the current order, on which functions will be computed. The window is defined by start and end integer arguments, with 0 denoting the current row, $-n$ and $+n$ denote the n -th element before or after
 - `rangeBetween` defines a window of rows, in the sequence of values, with similar mechanisms

Functions on windows: Example

Find the average concentration by station in Bologna or Rimini in sliding windows of size 5.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import desc
dfrb = df.where("Station = 'Rimini' or Station = 'Bologna'")
win = Window.partitionBy("Station").orderBy(asc("Date")).rowsBetween(-4, 0)
dfrb.select(col("Station"), col("Date"),\
            avg(col("Concentration")).\
            over(win).alias("Avg conc")).show()
```

Station	Date	Avg conc
Bologna	2015-01-29	54.0
Bologna	2015-03-10	75.5
...	...	...
Rimini	2015-12-31	44.6

Rollup

The **rollup** operation computes aggregates in groups defined by a set of columns, and a chain of its subsets (for example, {Pollutant, Station}, {Pollutant}, $\emptyset$)

```
conc_totals = df.rollup("Pollutant", "Station")\
    .agg(max("Concentration"))\
    .select("Pollutant", "Station", "max(Concentration)")\
    .orderBy("Station", "Pollutant")
conc_totals.show()
```

Pollutant	Station	max(Concentration)
null	null	100
Benzene	null	100
Nitrogen dioxide	null	97
...	...	...
Particulate matter	Rimini	94

Cube

The **cube** operation computes aggregates in all groups defined by all subsets of a set of columns

```
conc_cube = df.cube("Pollutant", "Station")\
    .agg(max("Concentration"))\
    .select("Pollutant", "Station", "max(Concentration)")\
    .orderBy("Station", "Pollutant")
conc_cube.show()
```

Pollutant	Station	max(Concentration)
		100
Benzene		100
Nitrogen dioxide		97
Ozone		98
Particulate matter		97
	Bologna	98
...	...	...
	Cesena	83
...	...	...
Particulate matter	Rimini	94

Boolean expression

With Boolean expressions, alternative forms are available

- Expressions or columns

```
from pyspark.sql.functions import col
df.where("Station = 'Bologna'").show()
df.where(col("Station") = "Bologna").show()
df.where("Station <> 'Bologna'").show()    # inequality SQL op
df.where(col("Station") ≠ "Bologna").show()
```

- Expressing conjoined booleans serially

```
from pyspark.sql.functions import instr
conc_filter = col("Concentration") > 50
poll_filter = instr(df.Pollutant, "Particulate") ≥ 1
df.where(df.Station.isin("Bologna", "Rimini"))\
    .where(conc_filter | poll_filter).show()
```

Boolean columns

One can achieve the same effect by defining Boolean columns using `withColumn` and a `where` filter

```
from pyspark.sql.functions import instr
loc_filter = col("Station").isin("Bologna", "Rimini")
conc_filter = col("Concentration") > 50
poll_filter = instr(df.Pollutant, "Particulate") ≥ 1
df.withColumn("monitor", \
    loc_filter & (conc_filter | poll_filter))\
    .where("monitor").show(3)
```

Code	Concentration	Date	Pollutant	Station	monitor
88118	79	2015-10-30	Ozone	Rimini	true
190115	59	2015-02-04	Ozone	Rimini	true
102991	41	2015-04-02	Particulate matter	Rimini	true

String case

```
from pyspark.sql.functions import initcap, lower, upper
df.select(initcap(col("Pollutant")), \
          lower(col("Pollutant")), \
          upper(col("Pollutant"))).show(3)
```

initcap(Pollutant)	lower(Pollutant)	upper(Pollutant)
Ozone	ozone	OZONE
Particulate Matter	particulate matter	PARTICULATE MATTER
Ozone	ozone	OZONE

Space trimming

- In dirty data, spurious spaces often trail or lead a string
- `ltrim`, `rtrim`, `trim` from `pyspark.sql.functions` left- or right- trim a string, or trim both at beginning and end

```
from pyspark.sql.functions import lit, ltrim, rtrim, trim
df.select(ltrim(lit("    Ozone")).alias("lt")).show(1)
df.select(rtrim(lit("Ozone    ")).alias("rt")).show(1)
df.select(trim(lit("  Ozone  ")).alias("t")).show(1)
```

```
+-----+
|    lt|
+-----+
|Ozone|
+-----+
```

```
+-----+
|    rt|
+-----+
|Ozone|
+-----+
```

```
+-----+
|     t|
+-----+
|Ozone|
+-----+
```

Character padding

- In other cases, we might need to pad a string with leading or trailing spaces, or any character string, using `lpad` or `rpadd`

```
from pyspark.sql.functions import lit, lpad, rpad
df.select(lpad(lit("Ozone"), 10, "--").alias("lp")).show(1)
df.select(rpad(lit("Ozone"), 10, "--").alias("rp")).show(1)
```

```
+-----+
|      lp|
+-----+
|--Ozone|
+-----+
```

```
+-----+
|      rp|
+-----+
|Ozone--|
+-----+
```

Regular expressions

- We might want to extract from a text all strings belonging to a large set, or replace them
- **Regular expressions** (r. e. in short) are special strings which represent sets of strings. They can be used to represent the strings that must be extracted or replaced
- Any non-special character represents itself
 - Special characters in r. e. are `+ * . [ ] | ^ $ { }`
- `.` represents any character
- `[cc' ... ]` represents any of the characters `c, c', ...`
- `[^cc' ... ]` represents any character except `c, c', ...`
- `[c-c']` represents any character between `c` and `c'` in the character ordering

Regular expressions (cont'd)

- Assume r, r' stand for regular expressions and $\gamma, \gamma_1, \gamma_2, \gamma_3, \dots$, and $\gamma', \gamma'_1, \gamma'_2, \gamma'_3, \dots$ denote any string represented by r or r' , respectively
- rr' represents $\gamma\gamma'$. For example, $[ab]c$ represents ac or bc
- $r|r'$ represents γ or γ'
- r^+ represents $\gamma_1, \gamma_1\gamma_2, \gamma_1\gamma_2\gamma_3, \dots$; for example, $[ab]^+$ represents $a, b, aa, bb, ab, ba, aaa, aab, bba, bbb, \dots$
- $r\{m, n\}$ represents $\gamma_1\gamma_2 \dots \gamma_m, \gamma_1\gamma_2 \dots \gamma_m\gamma_{m+1}, \dots, \gamma_1\gamma_2 \dots \gamma_{n-1}\gamma_n$. For example, $[ab]\{1, 2\}$ represents a, b, aa, bb, ab, ba
- r^* represents the same set represented by r^+ and the empty string
- $^{\wedge}r$ and $r^{\$}$ represent γ at the beginning of a line or at the end of a line, respectively

Regular expressions example

- Reuters tweets from UCI's Health News in Twitter Data Set

```
from pyspark.sql.types import StructField, StructType,\
    StringType
from pyspark.sql.functions import regexp_extract
source = "Spark/Data/Health-Tweets/reuters_health.txt"
TweetSchema = StructType([
    StructField("Tweet Text", StringType(), True)
])
h_news = spark.read.format("text").schema(TweetSchema)\
    .load(source)
re = "H[1-9]N[1-9]| +flu +|^flu +| +flu$"
h_news.select(regexp_extract(col("Tweet Text"), re, 0)\
    .alias("Flu text")).where(length("Flu text") > 0)\
    .distinct().show()
```

Regular expressions example (cont'd)

- `regexp_extract` found and extracted into a new column all unique occurrences of 4-character flu virus codes, or the whole word `flu`.

We searched for

- H, and N, each followed by any character from 1 to 9
- `flu`, with one or more spaces on both sides
- `flu` at the beginning, followed by one or more spaces, or `flu` at the end, preceded by one or more spaces

```
+-----+
|Flu text|
+-----+
|   H7N3|
|   flu |
|   H7N9|
|   H1N1|
|   H5N1|
|   H5N8|
|   H5N6|
+-----+
```

Date and Time

```
from pyspark.sql.functions import current_date,\
    current_timestamp
curr_dt = spark.range(4)\
    .withColumn("curr date", current_date())\
    .withColumn("curr time", current_timestamp())
curr_dt.show()
```

id	curr date	curr time
0	2020-11-08	2020-11-08 16:47:...
1	2020-11-08	2020-11-08 16:47:...
2	2020-11-08	2020-11-08 16:47:...
3	2020-11-08	2020-11-08 16:47:...

Date operations

Adding or subtracting days

```
from pyspark.sql.functions import date_add, date_sub
yest_tom = curr_dt.where(col("id") = 1)\
    .select(date_sub(col("curr date"), 1)\
        .alias("Yesterday"),\
        date_add(col("curr date"), 1).alias("Tomorrow"))
```

Yesterday	Tomorrow
2020-11-07	2020-11-09

Date intervals

Computing the interval between two dates, in days or months

```
from pyspark.sql.functions import datediff, months_between, to_date
yest_tom.select(datediff(col("Tomorrow"), col("Yesterday"))).show()
yest_tom_2m = yest_tom.withColumn("2m from now", date_add(current_date(), 2*31))
yest_tom_2m.select(months_between(col("Tomorrow"), col("2m from now"))).show()
yest_tom_2m.select(months_between(col("Yesterday"), col("2m from now"))).show()
```

```
+-----+
|datediff(Tomorrow, Yesterday)|
+-----+
|                               2|
+-----+

+-----+
|months_between(Tomorrow, 2m from now, true)|
+-----+
|                                           -2.0|
+-----+

+-----+
|months_between(Yesterday, 2m from now, true)|
+-----+
|                                           -2.06451613|
+-----+
```

Date and timestamp conversions

Functions `to_date` and `to_timestamp` accept and require (respectively) a format argument, mismatches generate nulls

```
yest_tom.select(to_timestamp(lit("2020-20-11 10:00:01"), "yyyy-dd-MM hh:mm:ss")).show()  
yest_tom.select(to_date(lit("2020-12-20"), "yyyy-MM-dd")).show()  
yest_tom.select(to_date(lit("2020-13-20"), "yyyy-MM-dd")).show()
```

```
+-----+  
|to_timestamp('2020-20-11 10:00:01', 'yyyy-dd-MM hh:mm:ss')|  
+-----+  
|                                                                |  
|                                2020-11-20 10:00:01|  
+-----+  
  
+-----+  
|to_date('2020-12-20', 'yyyy-MM-dd')|  
+-----+  
|                                2020-12-20|  
+-----+  
  
+-----+  
|to_date('2020-13-20', 'yyyy-MM-dd')|  
+-----+  
|                                null|  
+-----+
```

Null values

In a DataFrame Row, a null value is inserted as None

```
from pyspark.sql import Row
from pyspark.sql.types import StructField, StructType, StringType,\
    LongType, IntegerType
r1 = Row(1, "2019-10-01", "Rome", 0, None)
r2 = Row(2, "2019-10-02", "Milan", None, 15)
r3 = Row(3, "2019-10-01", "Naples", 200, 17)
temp_schema = StructType([StructField("Id", StringType(), False),
                           StructField("Date", StringType(), True),
                           StructField("City", StringType(), True),
                           StructField("Altitude", IntegerType(), True),
                           StructField("Temperature", LongType(), True)])
air_temp = spark.createDataFrame([r1, r2, r3], temp_schema)
air_temp.show()
```

Id	Date	City	Altitude	Temperature
1	2019-10-01	Rome	0	null
2	2019-10-02	Milan	null	15
3	2019-10-01	Naples	200	17

Filling nulls

The DataFrame `na.fill` function replaces all null values of specified columns with non-null user values

```
air_temp = air_temp.na.fill({"Altitude":0, "Temperature":15})  
air_temp.show()
```

Id	Date	City	Altitude	Temperature
1	2019-10-01	Rome	0	15
2	2019-10-02	Milan	0	15
3	2019-10-01	Naples	200	17

Eliminating rows with nulls

`na.drop("any")` removes rows if they contain some null
`na.drop("all")` removes rows if they contain only nulls

```
from pyspark.sql import Row
from pyspark.sql.types import StructField, StructType, StringType,\
    LongType, IntegerType
r1 = Row("2019-10-01", "Rome", 0, None)
r2 = Row(None, None, None, None)
r3 = Row("2019-10-01", "Naples", 200, 17)
temp_schema = StructType([StructField("Date", StringType(), True),
                           StructField("City", StringType(), True),
                           StructField("Altitude", IntegerType(), True),
                           StructField("Temperature", LongType(), True)])
spark.createDataFrame([r1, r2, r3], temp_schema).na.drop("any").show()
spark.createDataFrame([r1, r2, r3], temp_schema).na.drop("all").show()
```

Date	City	Altitude	Temperature
2019-10-01	Naples	200	17

Date	City	Altitude	Temperature
2019-10-01	Rome	0	null
2019-10-01	Naples	200	17

Complex values as struct columns

Create complex values by parenthesised expressions

- The type of the resulting column is struct

```
at = air_temp.selectExpr("(Date, City) as TimeLoc", "Altitude", \
                          "Temperature")
at.show()
at.printSchema()
```

	TimeLoc	Altitude	Temperature
[2019-10-01, Rome]		0	15
[2019-10-02, Milan]		0	15
[2019-10-01, Naples]	200		17

```
root
|-- TimeLoc: struct (nullable = false)
|   |-- Date: string (nullable = true)
|   |-- City: string (nullable = true)
|-- Altitude: integer (nullable = false)
|-- Temperature: long (nullable = false)
```

Struct

The struct function takes column arguments and creates a struct column with them

```
at1 = air_temp.select(struct("Date", "City").alias("TimeLoc"),\n                          "Altitude", "Temperature")\n\nat1.show()\nat1.printSchema()
```

TimeLoc	Altitude	Temperature
[2019-10-01, Rome]	0	15
[2019-10-02, Milan]	0	15
[2019-10-01, Naples]	200	17

```
root
|-- TimeLoc: struct (nullable = false)
|   |-- Date: string (nullable = true)
|   |-- City: string (nullable = true)
|-- Altitude: integer (nullable = false)
|-- Temperature: long (nullable = false)
```

Struct columns in queries

Refer to components of struct columns either using dot notation in expressions, or using the `getField` method of columns

```
from pyspark.sql.functions import col
at.where("TimeLoc.City = 'Rome'")\
    .select(col("TimeLoc").getField("Date"), "Temperature")\
    .show()
```

TimeLoc.Date	Temperature
2019-10-01	15

Regular expression groups

Tweet data row: id, timestamp, text & URL link, separated by |

- Use r. e. groups, delimited by (), and group number 3 in regexp_extract to extract the tweet text
- Special character | has to be escaped as \|, whereas we want it to match itself

```
from pyspark.sql.functions import regexp_extract
re = "^[([0-9]+)\\|([a-zA-Z0-9: + ]+)\\|(.+)( http://reut.rs.*)$"
h_news_tw = h_news.select(\
    regexp_extract(col("Tweet Text"), re, 2).alias("Time"),
    regexp_extract(col("Tweet Text"), re, 3).alias("Tweet"))
h_news_tw.show(2)
```

	Time	Tweet
Thu Apr 09 21:40:...	Los Angeles close...	
Thu Apr 09 21:24:...	U.S. cuts poultry...	

Arrays

array values are useful for manipulating text columns

- Reuters tweets to arrays of words, using splitting

```
from pyspark.sql.functions import split
h_news_words = h_news_tw.select(\
    "Time", split(col("Tweet"), " ").alias("Tweet words"))
h_news_words.show(4)
```

Time	Tweet words
Thu Apr 09 21:40:...	[Los, Angeles, cl...
Thu Apr 09 21:24:...	[U.S., cuts, poul...
Thu Apr 09 21:04:...	[Fears, over, Rou...
Thu Apr 09 19:49:...	[Liberia, watchdo...

Explode

Puts each value of an array on a separate row, duplicating the values of the other columns

```
from pyspark.sql.functions import explode
h_news_cols = h_news_words.select(\
    "Time", explode(col("Tweet words")).alias("Word"))
h_news_cols.show(4)
```

Time	Word
Thu Apr 09 21:40:...	Los
Thu Apr 09 21:40:...	Angeles
Thu Apr 09 21:40:...	closes
Thu Apr 09 21:40:...	500

Transaction to normal form

```
from pyspark.sql import Row
from pyspark.sql.types import StructType, StructField, StringType,\
    IntegerType, ArrayType
from pyspark.sql.functions import explode, create_map
r1 = Row(1, ["milk", "biscuits"])
r2 = Row(2, ["pencil", "eraser"])
trans_schema = StructType([StructField("TransId",IntegerType(),False),
                             StructField("ItemSet",ArrayType(StringType(),True),True)])
trans_db = spark.createDataFrame([r1, r2], trans_schema)
trans_db.show()
trans_arr = trans_db.select("TransId", explode(col("ItemSet")).alias("Item"))
trans_arr.show()
```

TransId	ItemSet
1	[milk, biscuits]
2	[pencil, eraser]

TransId	Item
1	milk
1	biscuits
2	pencil
2	eraser

Maps

Function `create_map` creates key-value pairs from columns

```
from pyspark.sql.functions import create_map
exp_poll = df.select(create_map(col("Date"),
                                col("Concentration"))\
                    .alias("DateConc"))
exp_poll.show(3, False) # False: don't truncate
```

```
+-----+
|DateConc      |
+-----+
|[2015-10-30 -> 79]|
|[2015-10-08 -> 25]|
|[2015-02-04 -> 59]|
+-----+
```

Maps access

Maps can be accessed by key, using expressions and square brackets, or columns and square brackets, as in a dictionary access by key

- Non matching rows generate nulls

```
exp_poll.selectExpr("Date_conc['2015-10-30']").show(2)
```

```
exp_poll.select(col("Date_conc")['2015-10-30']).show(2)
```

```
+-----+
|Date_conc[2015-10-30]|
+-----+
|                      79|
|                      null|
+-----+
```

Maps and explode

The `explode` function is applicable to maps

- The resulting columns are named `key` and `value`
- Example: Transactions as maps to normal form

```
trans_map = trans_db.select(create_map(col("TransId"),\
                                       col("ItemSet"))\
                           .alias("IdSet"))

trans_map.show()

trans_nf1 = trans_map.select(explode(col("IdSet")))\
                        .select("key",explode("value"))

trans_nf1.show()
```

```
+-----+
|               IdSet|
+-----+
|[1 -> [milk, bisc...|
|[2 -> [pencil, er...|
+-----+
```

```
+---+-----+
|key|    col|
+---+-----+
| 1|   milk|
| 1|biscuits|
| 2|  pencil|
| 2|  eraser|
+---+-----+
```

Section 6

MLlib

Vector type

In both supervised and unsupervised machine learning, features are encoded as vectors, dense or sparse

- Sparse vectors are defined by elements, their number, and their positions

```
from pyspark.ml.linalg import Vectors
v = Vectors.dense(3.141, 2.718)
s = Vectors.sparse(3, [1, 2], [3.141, 2.718])
v
s
```

```
DenseVector([3.141, 2.718])
```

```
SparseVector(3, {1: 3.141, 2: 2.718})
```

Reading data into DataFrame

- `spark.read` is a `DataFrameReader` for reading data into a `DataFrame`, with methods
- `format` : file format, such as CSV, JSON, plain text
- `schema` : schema of the data
- `option` : multiple options dependent on the format, such as behaviour when the data are malformed, or whether to infer the schema or not
- Each method returns a `DataFrameReader`
- Methods are optional and can be chained

CSV file read: Wine dataset

```
from pyspark.sql.types import StructType, StructField, IntegerType, DoubleType
wine_schema = StructType([\
    StructField("Class", IntegerType(), True),\
    StructField("Alcohol", DoubleType(), True),\
    StructField("Malic acid", DoubleType(), True),\
    StructField("Ash", DoubleType(), True),\
    StructField("Alcalinity of ash", DoubleType(), True),\
    StructField("Magnesium", DoubleType(), True),\
    StructField("Total phenols", DoubleType(), True),\
    StructField("Flavonoids", DoubleType(), True),\
    StructField("Nonflavonoid phenols", DoubleType(), True),\
    StructField("Proanthocyanins", DoubleType(), True),\
    StructField("Color intensity", DoubleType(), True),\
    StructField("Hue", DoubleType(), True),\
    StructField("OD280/OD315 of diluted wines", DoubleType(), True),\
    StructField("Proline", DoubleType(), True)])

wine = spark.read.format("csv")\
    .schema(wine_schema)\
    .option("sep", ",")\
    .option("path", "Spark/Data/wine/wine.data").load()
```

Model input preparation

```
from pyspark.ml.feature import RFormula
rf = RFormula(formula="class ~ .")
rf_fit = rf.fit(wine)
rf_transfd = rf_fit.transform(wine)
rf_transfd.printSchema()
```

```
root
|-- Class: integer (nullable = true)
|-- Alcohol: double (nullable = true)
|-- Malic acid: double (nullable = true)
|-- Ash: double (nullable = true)
|-- Alcalinity of ash: double (nullable = true)
|-- Magnesium: double (nullable = true)
|-- Total phenols: double (nullable = true)
|-- Flavonoids: double (nullable = true)
|-- Nonflavonoid phenols: double (nullable = true)
|-- Proanthocyanins: double (nullable = true)
|-- Color intensity: double (nullable = true)
|-- Hue: double (nullable = true)
|-- OD280/OD315 of diluted wines: double (nullable = true)
|-- Proline: double (nullable = true)
|-- features: vector (nullable = true)
```

Model fit: Trees on Wine

```
from pyspark.ml.classification import DecisionTreeClassifier
train, test = rf_transfd.randomSplit([0.7, 0.3])
in_train = train.selectExpr("features", "cast(class as double) as label")
tree = DecisionTreeClassifier()
print(tree.explainParams())
tree_mod = tree.fit(in_train)
tree_mod.transform(in_train).select("label", "prediction").show()
```

Wine quality dataset: CSV read

Semi-colon separated with a header line

```
wineq = spark.read.format("csv")\  
  .option("sep", ";")\  
  .option("header", "True")\  
  .option("inferSchema", "True")\  
  .option("path", "Spark/Data/winequality/winequality-red.csv").load()
```

```
root  
|-- fixed acidity: double (nullable = true)  
|-- volatile acidity: double (nullable = true)  
|-- citric acid: double (nullable = true)  
|-- residual sugar: double (nullable = true)  
|-- chlorides: double (nullable = true)  
|-- free sulfur dioxide: double (nullable = true)  
|-- total sulfur dioxide: double (nullable = true)  
|-- density: double (nullable = true)  
|-- pH: double (nullable = true)  
|-- sulphates: double (nullable = true)  
|-- alcohol: double (nullable = true)  
|-- quality: integer (nullable = true)
```

Wine quality dataset: Tree model

```
from pyspark.ml.feature import RFormula
from pyspark.ml.classification import DecisionTreeClassifier
rfwq = RFormula(formula="quality ~ .")
rf_fit_wq = rfwq.fit(wineq)
rf_transfd_wq = rf_fit_wq.transform(wineq)
train_wq, test_wq = rf_transfd_wq.randomSplit([0.7, 0.3])
in_train_wq = train_wq.selectExpr("features", "label")
tree_wq = DecisionTreeClassifier()
tree_mod_wq = tree_wq.fit(in_train_wq)
tree_mod_wq.transform(in_train_wq).select("label", "prediction")\
    .show(4)
```

```
+-----+-----+
|label|prediction|
+-----+-----+
| 4.0|      5.0|
| 6.0|      6.0|
| 7.0|      6.0|
| 6.0|      6.0|
+-----+-----+
```

Sonar data

- Sonar signals labeled as "R", bounced by an approximately cylindrical rock, or "M", bounced by a cylindrical metal object, that is, a mine
- Columns are 60 energy values at increasing frequencies, and the class label

```
rm = spark.read.format("csv").option("header", "False")\
    .option("inferSchema", "True")\
    .load("Spark/Data/connectionist-bench/sonar.all-data")\
    .withColumnRenamed("_c60", "class")
rm.select("_c0", "_c1", "_c2", "_c4", "class").show(3)
```

```
+-----+-----+-----+-----+-----+
|  _c0|  _c1|  _c2|  _c4|class|
+-----+-----+-----+-----+-----+
|  0.02|0.0371|0.0428|0.0954|    R|
|0.0453|0.0523|0.0843|0.1183|    R|
|0.0262|0.0582|0.1099|0.0974|    R|
+-----+-----+-----+-----+-----+
```

Abalone data

- Predict the age of abalones from sex and measurements
- 8 columns (sex, length and weight measurements) and the count of growth rings of the shell, which determines the age

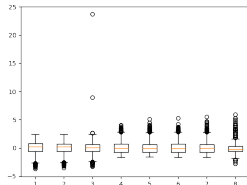
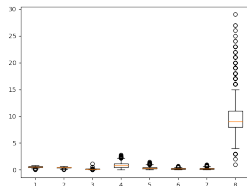
```
from pyspark.sql.types import StructType, StructField, IntegerType, DoubleType, \
    StringType
```

```
aba_schema = StructType([\
    StructField("Sex", StringType(), True),\
    StructField("Length", DoubleType(), True),\
    StructField("Diameter", DoubleType(), True),\
    StructField("Height", DoubleType(), True),\
    StructField("Whole", DoubleType(), True),\
    StructField("Shucked wt", DoubleType(), True),\
    StructField("Viscera wt", DoubleType(), True),\
    StructField("Shell wt", DoubleType(), True),\
    StructField("Rings", IntegerType(), True)])\
aba = spark.read.format("csv").option("header", "False").schema(aba_schema)\
    .option("inferSchema", "True").load("Spark/Data/abalone/abalone.data")
```

Abalone data (cont'd)

- Explore the data with Pandas and box plots and normalise

```
import matplotlib.pyplot as plt
aba_pd = aba.toPandas()
aba_num = aba_pd.iloc[:, 1:]
boxfig = plt.figure()
plt.boxplot(aba_num)
boxfig.savefig("Plots/abalone-box.png")
# normalise the data
desc_stats = aba_pd.describe()
aba_norm = aba_pd.iloc[:, 1:]
for c in desc_stats.columns:
    mean = desc_stats.loc['mean', c]
    sd = desc_stats.loc['std', c]
    aba_norm.loc[:, c] = (aba_pd.loc[:, c] - mean) / sd
aba_normarr = aba_norm.values
boxnfig = plt.figure()
plt.boxplot(aba_normarr)
boxnfig.savefig("Plots/abalone-norm-box.png")
```

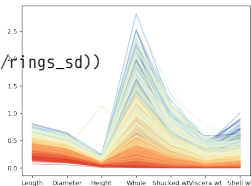


Abalone parallel plots

Draw one series of connected straight line segments for each observation

- Connecting points are column values
- The colour of the line is proportional to the target
- normalise and transform with $\frac{1}{1+\exp(-x)}$

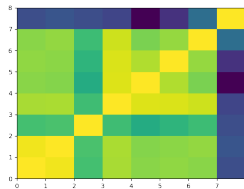
```
from numpy import exp
rings_mean = desc_stats.loc['mean', 'Rings']
rings_sd = desc_stats.loc['std', 'Rings']
colour = 1.0/(1.0 + exp(-(aba_pd['Rings'].values - rings_mean)/rings_sd))
parfig = plt.figure()
for i in range(len(aba_pd)):
    row = aba_pd.iloc[i, 1:len(aba_pd.columns)-1]
    row.plot(color=plt.cm.RdYlBu(colour[i]), alpha=.4)
parfig.savefig("Plots/abalone-par.png")
```



Abalone correlation

Heat map with colour shades proportional to the correlation between columns

```
corr = aba_pd.iloc[:, 1:9].corr()  
print(corr)  
heatfig = plt.figure()  
plt.pcolor(corr)  
heatfig.savefig("Plots/abalone-heat.png")
```



	Length	Diameter	Height	Whole	Shucked wt	Viscera wt	Shell wt	Rings
Length	1.000000	0.986812	0.827554	0.925261	0.897914	0.903018	0.897706	0.556720
Diameter	0.986812	1.000000	0.833684	0.925452	0.893162	0.899724	0.905330	0.574660
Height	0.827554	0.833684	1.000000	0.819221	0.774972	0.798319	0.817338	0.557467
Whole	0.925261	0.925452	0.819221	1.000000	0.969405	0.966375	0.955355	0.540390
Shucked wt	0.897914	0.893162	0.774972	0.969405	1.000000	0.931961	0.882617	0.420884
Viscera wt	0.903018	0.899724	0.798319	0.966375	0.931961	1.000000	0.907656	0.503819
Shell wt	0.897706	0.905330	0.817338	0.955355	0.882617	0.907656	1.000000	0.627574
Rings	0.556720	0.574660	0.557467	0.540390	0.420884	0.503819	0.627574	1.000000

Wine quality data

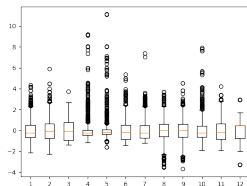
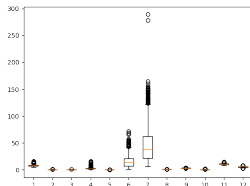
- Predict the evaluation of wine quality by experts from physical and chemical properties
- 11 numeric columns, and the numeric score given by at least 3 experts

```
from pyspark.sql.types import StructType, StructField, DoubleType
wineq_schema = StructType([\
    StructField("FAcid", DoubleType(), True),      # 1 - fixed acidity
    StructField("VAcid", DoubleType(), True),      # 2 - volatile acidity
    StructField("Citr", DoubleType(), True),        # 3 - citric acid
    StructField("ReSug", DoubleType(), True),       # 4 - residual sugar
    StructField("Chlo", DoubleType(), True),        # 5 - chlorides
    StructField("FrSO2", DoubleType(), True),       # 6 - free sulfur dioxide
    StructField("TSO2", DoubleType(), True),       # 7 - total sulfur dioxide
    StructField("Den", DoubleType(), True),         # 8 - density
    StructField("pH", DoubleType(), True),          # 9 - pH
    StructField("Sulph", DoubleType(), True),       # 10 - sulphates
    StructField("Alc", DoubleType(), True),         # 11 - alcohol
    StructField("Qual", DoubleType(), True)])       # 12 - quality (score between 0 and 10)
wineq = spark.read.format("csv").option("header", "True").option("sep", ";")\
    .schema(wineq_schema).option("inferSchema", "True")\
    .load("Spark/Data/winequality/winequality-red.csv")
```

Wine quality data (cont'd)

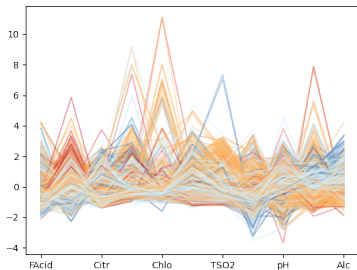
- Explore the data with box plots in Pandas, then normalise

```
import matplotlib.pyplot as plt
wineq_pd = wineq.toPandas()
# wineq_arr = wineq_pd.iloc[:, :len(wineq_pd.columns)]
boxfig = plt.figure()
plt.boxplot(wineq_pd)
boxfig.savefig("Plots/winequality-box.png")
# normalise the data
desc_stats = wineq_pd.describe()
wineq_norm = wineq_pd.copy()
for c in desc_stats.columns:
    mean = desc_stats.loc['mean', c]
    sd = desc_stats.loc['std', c]
    wineq_norm.loc[:, c] = (wineq_pd.loc[:, c] - mean) / sd
# wineq_normarr = wineq_norm.values
boxnfig = plt.figure()
plt.boxplot(wineq_norm)
boxnfig.savefig("Plots/winequality-norm-box.png")
```



Wine quality parallel plots

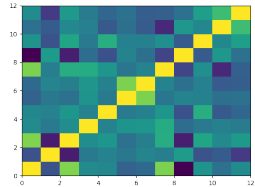
```
import numpy as np
from numpy import exp
colour = 1.0/(1.0 + exp(-wineq_norm['Qual'].values))
parfig = plt.figure()
cols = wineq_norm.columns.tolist()[:-1]
for i in range(len(wineq_norm)):
    row = wineq_norm.iloc[i, :len(wineq_norm.columns)-1]
    plt.xticks(np.arange(len(row)), cols, rotation=85)
    row.plot(color=plt.cm.RdYlBu(colour[i]), alpha=.4)
parfig.savefig("Plots/winequality-par.png")
```



Wine quality correlation

Heat map with colour shades proportional to correlation

```
corr = wineq_pd.corr()
print(corr)
heatfig = plt.figure()
plt.pcolor(corr)
heatfig.savefig("Plots/winequality-heat.png")
```



	FAcid	VAcid	Citr	ReSug	Chlo	FrS02	TS02	Den	pH	Sulph	Alc	Qual
FAcid	1.000000	-0.256131	0.671703	0.114777	0.093705	-0.153794	-0.113181	0.668047	-0.682978	0.183006	-0.061668	0.124052
VAcid	-0.256131	1.000000	-0.552496	0.001918	0.061298	-0.010504	0.076470	0.022026	0.234937	-0.260987	-0.202288	-0.390558
Citr	0.671703	-0.552496	1.000000	0.143577	0.203023	-0.060978	0.035533	0.364947	-0.541904	0.312770	0.109903	0.226373
ReSug	0.114777	0.001918	0.143577	1.000000	0.055610	0.187049	0.203028	0.355283	-0.005652	0.005527	0.042075	0.013732
Chlo	0.093705	0.061298	0.203023	0.055610	1.000000	0.005562	0.047400	0.200632	-0.265026	0.371260	-0.221141	-0.128907
FrS02	-0.153794	-0.010504	-0.060978	0.187049	0.005562	1.000000	0.667666	-0.021946	0.070377	0.051658	-0.069408	-0.050656
TS02	-0.113181	0.076470	0.035533	0.203028	0.047400	0.667666	1.000000	0.071269	-0.066495	0.042947	-0.205654	-0.185100
Den	0.668047	0.022026	0.364947	0.355283	0.200632	-0.021946	0.071269	1.000000	-0.341699	0.148506	-0.496180	-0.174919
pH	-0.682978	0.234937	-0.541904	-0.005652	-0.265026	0.070377	-0.066495	-0.341699	1.000000	-0.196648	0.205633	-0.057731
Sulph	0.183006	-0.260987	0.312770	0.005527	0.371260	0.051658	0.042947	0.148506	-0.196648	1.000000	0.093595	0.251397
Alc	-0.061668	-0.202288	0.109903	0.042075	-0.221141	-0.069408	-0.205654	-0.496180	0.205633	0.093595	1.000000	0.476166
Qual	0.124052	-0.390558	0.226373	0.013732	-0.128907	-0.050656	-0.185100	-0.174919	-0.057731	0.251397	0.476166	1.000000

Section 7

Spark unstructured API

Resilient Distributed Dataset (RDD)

Collection of objects

- Read only
- Divided into **partitions**
- It is created with deterministic operations from
 - Data in persistent memory
 - Other RDDs
 - Collections of objects in the driver program
- Not necessarily materialized
 - Associated with a **lineage** that represents the operations sufficient to build it from the data in persistent memory
- User-controlled persistence and partitioning
 - RDD used repeatedly can be made persistent
 - Partitioning can be decided to maximize the location of data relative to computation

Processing with RDD

- **Creation:** produces an RDD
 - Input:
 - File
 - HDFS
 - Other RDDs
 - Output: RDD
- **Transformation**
 - Input: RDD
 - Output: RDD
 - Examples: filtering, application of function, join
- **Action**
 - Input: RDD
 - Output: returns a value to the application, or exports data to the mass memory
 - Examples: counting, average, retrieval of the set of values, saving

parallelize

Creates an RDD by distributing the contents of the parameter

```
>>> rdd = sc.parallelize([2, 3, 5, 7,  
...                       11, 13, 17, 19,  
...                       23, 29, 31, 37,  
...                       41, 43, 47, 53])
```

```
>>> rdd
```

```
ParallelCollectionRDD[1] at parallelize at PythonRDD.scala:475
```

```
>>> type(rdd)
```

```
<class 'pyspark.rdd.RDD'>
```

collect

Access the elements of an RDD and display them

```
>>> rdd.collect()  
[2, 3, 5, 7, 11, 13, 17, 19,  
 23, 29, 31, 37, 41, 43, 47, 53]
```

Use the method only if the size of the RDD is small!

take, first

Take (n) returns a list containing the first n elements of an RDD

First () returns the first element

```
>>> rdd.take(4)
```

```
[2, 3, 5, 7]
```

```
>>> rdd.first()
```

```
2
```

count

Returns the number of elements in an RDD

```
>>> rdd.count()
```

```
16
```

min, max

Return the minimum and maximum of an RDD

```
>>> ds = sc.parallelize(['c', 'b', 'a', 'd'])
```

```
>>> ds.min(); ds.max()
```

```
'a'
```

```
'd'
```

variance, stdev

variance, stdev return the variance and standard deviation of the elements of an RDD

```
>>> ds = sc.parallelize([-2, -1, 0, 1, 2])
```

```
>>> ds.variance(); ds.stdev()
```

```
2.0
```

```
1.4142135623730951
```

sampleVariance, sampleStdev

sampleVariance, sampleStdev return the unbiased variance and deviation

```
>>> ds.sampleVariance()
```

```
2.5
```

```
>>> ds.sampleStdev()
```

```
1.5811388300841898
```

map

Applies a function to each element of an RDD

```
>>> f = sc.textFile("measurements.csv")
>>> f.take(2)
['88118,2015-10-30,Ozone,Rimini,79',
 '927170,2015-10-08,Particulate matter,Ravenna,25']
>>> f.map(lambda l: l.split(",")).take(2)
[['88118', '2015-10-30', 'Ozone',
  'Rimini', '79'],
 ['927170', '2015-10-08', 'Particulate matter',
  'Ravenna', '25']]
```

mapValues

Applies a function to the values of the elements of an RDD without changing keys or partitioning

The value of an element of an RDD is the second component

```
>>> f = sc.textFile("measurements.csv")
>>> f.take(2)
['88118,2015-10-30,Ozone,Rimini,79',
 '927170,2015-10-08,Particulate matter,Ravenna,25']
>>> f.map(lambda l: l.split(",")).take(2)
[['88118', '2015-10-30', 'Ozone',
  'Rimini', '79'],
 ['927170', '2015-10-08', 'Particulate matter',
  'Ravenna', '25']]
>>> f.map(lambda l: l.split(",")).mapValues(lambda x: x[:4]).take(2)
[('88118', '2015'), ('927170', '2015')]
```

flatMap

Applies a function to each element of an RDD, then “flattens” the result, that is, eliminates nesting preserving values

```
>>> rdd = sc.parallelize([2, 3, 4])  
>>> sorted(rdd.flatMap(lambda x: range(1, x)).collect())  
[1, 1, 1, 2, 2, 3]  
>>> sorted(rdd.flatMap(lambda x: [(x, x), (x, x)]).collect())  
[(2, 2), (2, 2), (3, 3), (3, 3), (4, 4), (4, 4)]
```

(cf. `flatMap` in Spark Python API)

zip

Transposes two RDDs, concatenating the elements with the same position

```
>>> benchmarks1 = sc.parallelize(
...     [1.83, 7.56, 7.30, 5.20, 8.95])
>>> benchmarks2 = sc.parallelize(
...     [27.54, 57.48, 60.86, 33.25, 36.34])
>>> benchmarks = benchmarks1.zip(benchmarks2)
>>> benchmarks.collect()
[(1.83, 27.54), (7.56, 57.48), (7.3, 60.86),
(5.2, 33.25), (8.95, 36.34)]
```

zipWithIndex

Associates with each element of a RDD a progressive integer number, starting at 0. Equivalent to `zip()` with `0, 1, 2, 3, ...`

```
>>> benchmarksI = benchmarks.zipWithIndex()
>>> benchmarksI.collect()
[((1.83, 27.54), 0), ((7.56, 57.48), 1),
((7.3, 60.86), 2), ((5.2, 33.25), 3),
((8.95, 36.34), 4)]
```

takeSample

Returns a fixed cardinality sample of an RDD First argument: `True` → replacement, `False` → without replacement

Suitable for small cardinality samples

```
>>> f = sc.textFile("measurements.csv")
>>> fs = f.map(lambda l: l.split(","))
>>> fs.takeSample(False, 3)
[['238596', '2015-10-28', 'Particulate matter',
  'Modena', '83'],
 ['460813', '2015-10-18', 'Ozone',
  'Bologna', '58'],
 ['551808', '2015-03-04', 'Particulate matter',
  'Rimini', '34']]
```

filter

Select the elements of an RDD that make an expression true

```
>>> f = sc.textFile("measurements.csv")
>>> fs = f.map(lambda l: l.split(","))
>>> ff = fs.filter(lambda x: x[1] ≥ '2015-03-01')
>>> ff.take(3)
[['88118', '2015-10-30', 'Ozone',
  'Rimini', '79'],
 ['927170', '2015-10-08', 'Particulate matter',
  'Ravenna', '25'],
 ['697379', '2015-12-13', 'Ozone',
  'Cesena', '0']]
```

keys

Returns an RDD containing the keys of the RDD passed as a parameter
The term "key" does not imply the assumption or imposition of any relational constraint

```
>>> rdd = sc.parallelize([[2.0, 5.0, 1.0],  
                          [43.0, 10.0, 1.5],  
                          [1., 0., 1.0],  
                          [-1., 100., 1.5],  
                          [2.0, 1.0, 1.0]])  
  
>>> rdd.keys().collect()  
[2.0, 43.0, 1.0, -1.0, 2.0]
```

intersection, union

```
>>> rdd = sc.parallelize([4, 5, 2, 6, 0, -1])  
>>> rdd1 = sc.parallelize([0, 2, 9, -1, 0, -10, 4])
```

- Intersection

```
>>> rdd.intersection(rdd1).collect()  
[0, 2, 4, -1]
```

- Union

```
>>> rdd.union(rdd1).collect()  
[4, 5, 2, 6, 0, -1, 0, 2, 9, -1, 0, -10, 4]
```

groupByKey

Returns an RDD of lists [k, v] in which v is the list of values of the elements with key k in the RDD that calls the method

```
>>> f = sc.textFile("measurements.csv")
>>> f1 = f.map(lambda l: l.split(","))
>>> f2 = f1.map(lambda x: [x[3], x[:3]+[x[4]]])
>>> f2.first()
['Rimini', ['88118', '2015-10-30', 'Ozone', '79']]
>>> f3 = f2.groupByKey().mapValues(list)
>>> f3.first()
('Rimini',
 [['88118', '2015-10-30', 'Ozone', '79'],
  ['190115', '2015-02-04', 'Ozone', '59']] ...)
```

cartesian

Returns the Cartesian product of two RDDs

```
>>> rdd = sc.parallelize([1, 2, 3])  
>>> rdd.cartesian(rdd).collect()  
[(1, 1), (1, 2), (1, 3), (2, 1), (2, 2),  
 (2, 3), (3, 1), (3, 2), (3, 3)]
```

Spark applications

- Interactive local use is a powerful tool for exploring, programming, and debugging
- In a production environment, one launches an already debugged Spark application on a cluster using the `spark-submit` command, with options

```
$SPARK_INSTALL/bin/spark-submit --master local myprog.py
```

Word count

word_count.py

```
from pyspark import SparkContext
def main():
    sc = SparkContext(appName='SparkWordCount')
    input_file = sc.textFile('/vagrant/Spark/WordCount/chaucer.txt')
    counts = input_file.flatMap(lambda line: line.split()) \
        .map(lambda word: (word, 1)) \
        .reduceByKey(lambda a, b: a + b)
    counts.saveAsTextFile('/vagrant/Spark/WordCount/output')
    sc.stop()
if __name__ == '__main__':
    main()
```

word_count.sh

```
#!/usr/bin/env bash
rm -rf output
$SPARK_INSTALL/bin/spark-submit --master local word_count.py
```